

GSM ToolBox

A user-friendly desktop application for genome-scale metabolic modelling
and metabolic engineering

The Complete User Manual

From opening a model to designing high-yield strains

Built on COBRApy · Powered by GLPK & SCIP · Interactive maps by Escher

How to use this manual

This manual is written for **biologists and metabolic engineers** who want to use genome-scale metabolic models without writing code. It is organised as a guided journey. If you are new to the field, read Part I first: it explains, in plain language, what a metabolic model is and how the mathematics behind it works, so the rest of the manual makes sense. If you already know the theory and just want to operate the software, start at Part II and use Part I as a reference.

The chapters follow the natural order of a project: you **open a model** and check it (Parts II–III), **set the growth condition and objective** (Part III), **explore the network** visually (Part IV), **run simulations** to characterise the strain (Parts V–VI), **design interventions** to raise the yield of a product (Parts VII–VIII), integrate **omics data** and study **communities** (Part IX), and finally follow a **complete worked example** that ties everything together (Part X). Appendices give a method-comparison table, a glossary, a troubleshooting FAQ, and the reference list.

Note. Look for shaded boxes throughout. **Why it matters** explains the biological or engineering purpose of a feature, **Tip** gives a practical shortcut, **Rule of thumb** offers default settings, and **Caution** warns about a common mistake or a limit of the method.

Table of Contents

How to use this manual	2
1. Introduction and key concepts	7
1.1 What is a genome-scale metabolic model?	7
1.2 From genes to phenotype: the gene–protein–reaction logic	7
1.3 Constraint-based modelling and the steady-state assumption	7
1.4 Flux Balance Analysis, explained	8
1.5 What you can and cannot conclude	9
1.6 A map of the toolbox	9
2. Installation and the interface	12
2.1 Launching the application	12
2.2 A tour of the window	12
2.3 The menus	12
3. Loading and inspecting a model	15
3.1 Opening a model	15
3.2 The Explorer: searching everything at once	15
3.3 Reading model quality before you trust it	15

4. Tweaking basic parameters	16
4.1 Editing a reaction	16
4.2 Adding a reaction: the reaction builder	16
4.3 Setting the growth medium	17
4.4 Choosing the optimisation objective	18
4.5 Saving, exporting and undo	19
5. Seeing the metabolic network	21
5.1 The Network Map	21
5.2 What the colours mean	21
5.3 Categories: grouping and isolating pathways	21
5.4 The Escher Visualizer: interactive maps and an FBA navigator	22
5.5 The Strategy Visualizer: comparing engineering rounds	23
6. Core flux simulations	25
6.1 Flux Balance Analysis (FBA)	25
6.2 Parsimonious FBA (pFBA)	25
6.3 Flux Variability Analysis (FVA)	25
6.4 Shadow prices and reduced costs	26
6.5 Mutant simulation (MOMA / ROOM)	26
7. Phenotype and yield analyses	28
7.1 Production envelope	28
7.2 Robustness analysis	29
7.3 Phenotypic phase plane (PhPP)	29
7.4 Flux sampling (distributions)	29
8. Essentiality and model curation	29
8.1 Gene and reaction deletion (essentiality)	30
8.2 The quality report	30
8.3 Blocked reactions	30
8.4 Gap-filling	30
9. Strain design	33
9.1 The design problem, in one picture	33
9.2 Knockout strain design (OptKnock / RobustKnock / OptCouple / Heuristic)	33
9.3 FSEOF: amplification and knock-down targets	33

9.4 Overproduce a metabolite	34
9.5 Reading and prioritising designs	34
10. Designing heterologous pathways	36
10.1 The idea	36
10.2 Reaction databases and the offline cache	36
10.3 Different databases, different identifiers. Handled automatically	36
10.4 Predicting a pathway	36
10.5 Inspecting, drawing and applying a route	37
10.6 From an EC number to a gene	38
11. Omics integration and energy analysis	41
11.1 Preparing an omics dataset	41
11.2 eFlux	41
11.3 GIMME	41
11.4 ATP-maintenance sensitivity	41
12. Communities and pathway architecture	42
12.1 Community (multi-organism) modelling	42
12.2 Elementary flux modes (EFMs)	42
13. A complete yield-improvement workflow	44
13.1 Characterise the baseline	44
13.2 Make sure the product is reachable	44
13.3 Find and commit interventions	45
13.4 Verify and compare rounds	45
14. Dynamic analysis: how the network responds as conditions change	47
14.1 Condition scan	47
14.2 Time course	47
14.3 Stored runs, tables and plots	47
14.4 Overlaying runs on one plot	47
15. Regulation: constraining a model with what the cell actually does	49
15.1 Rule sets are files you load	49
15.2 What a rule can do	49
15.3 Check a rule set before you trust it	49

15.4 The rule editor	49
16. Pathway design: rule-based chemistry, enzymes and feasibility	50
16.1 When the databases have no route	50
16.2 Finding an enzyme	50
16.3 Thermodynamic feasibility	50
16.4 Chemistry checks on a proposed route	50
17. Tasks, performance, results and settings	51
17.1 Background tasks and concurrency	51
17.2 A note on speed and CPU usage	51
17.3 Results folder and saving	51
17.4 Settings and the offline cache	51
Appendix A. Method comparison at a glance	53
Appendix B. Glossary	55
Appendix C. Troubleshooting and FAQ	56
Appendix D. File formats and where data is stored	57
Appendix E. Worked examples	58
E.1 A light-driven CO ₂ -to-styrene consortium	58
E.2 Integrating omics data to refine predictions	63
References	68

PART I

Foundations

What a genome-scale metabolic model is, and how the constraint-based mathematics turns that model into testable predictions. Read this once and the rest of the toolbox will feel intuitive.

1. Introduction and key concepts

1.1 What is a genome-scale metabolic model?

A **genome-scale metabolic model** (GSM, sometimes GEM) is a structured, computable inventory of (nearly) all the metabolic reactions an organism can perform. It is reconstructed from the genome: each gene that encodes a metabolic enzyme is linked to the reaction(s) that enzyme catalyses, and each reaction is written with its **stoichiometry**, which metabolites it consumes and produces, and in what proportions. A modern bacterial model contains on the order of one to two thousand reactions and metabolites; eukaryotic and plant models are larger.

Three ingredients make a reconstruction into a *predictive* model: the **stoichiometric matrix** (the reactions and their metabolites), the **flux bounds** (how fast each reaction may run, and in which direction), and a **biomass reaction**. A pseudo-reaction that drains precursors (amino acids, nucleotides, lipids, cofactors, energy) in the ratios needed to build one gram of new cell. Optimising flux through the biomass reaction is how the model predicts growth.

1.2 From genes to phenotype: the gene–protein–reaction logic

Each reaction carries a **gene–reaction rule** (GPR): a Boolean expression over the genes needed to catalyse it. *AND* means the gene products form a complex (all are required); *OR* means isozymes (any one suffices). GPRs are what let the model translate a genetic change into a metabolic one: deleting a gene removes the reactions whose GPR can no longer be satisfied, and the consequences ripple through the network. This is the basis of the gene-deletion and essentiality analyses in Part VI.

1.3 Constraint-based modelling and the steady-state assumption

We rarely know every enzyme's kinetic rate constant at genome scale, so GSMs avoid kinetics entirely. Instead they use **constraint-based modelling**: rather than computing one exact trajectory, they describe the *space of all flux distributions that are physically possible*, and then reason about that space. Two kinds of constraint define it. First, **mass balance at steady state**: for every internal metabolite, total production equals total consumption, so its concentration does not change. Written compactly this is $S \cdot v = 0$, where S is the stoichiometric matrix and v is the vector of reaction fluxes. Second, **capacity/thermodynamic bounds**: each flux lies between a lower and an upper bound (an irreversible reaction has a lower bound of zero; a limited nutrient uptake sets an upper bound).

The steady-state assumption: every internal metabolite is balanced

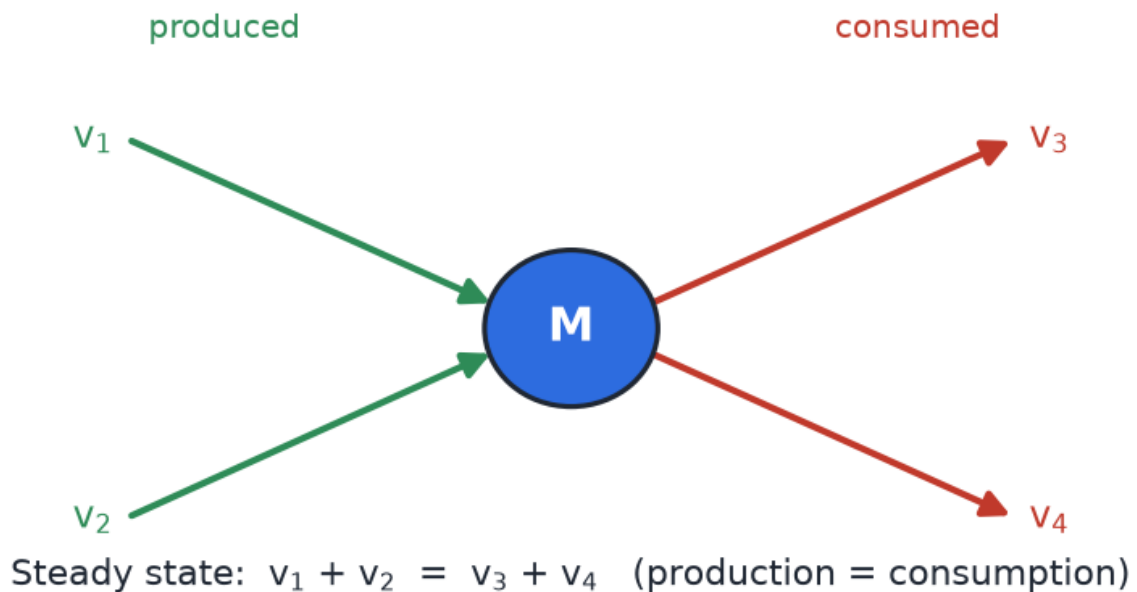


Figure 1.1. Steady state: for each internal metabolite, the fluxes that produce it sum to the fluxes that consume it, so its pool stays constant.

Why it matters. The steady-state assumption is what makes genome-scale prediction tractable without kinetic parameters. It holds well for cells in balanced (exponential) growth. It does *not* describe fast transients, storage dynamics, or regulation. Keep that in mind when interpreting results.

1.4 Flux Balance Analysis, explained

The constraints above usually leave many feasible flux distributions. A whole multi-dimensional space of them. **Flux Balance Analysis (FBA)** picks one by optimisation: it chooses an **objective** (a reaction to maximise or minimise, typically biomass) and uses **linear programming** to find the feasible flux distribution that pushes the objective as far as it will go [1]. Geometrically, the feasible space is a convex polytope and the optimum sits at one of its corners (Figure 1.2).

Constraint-based modelling: FBA finds the best state in a feasible space

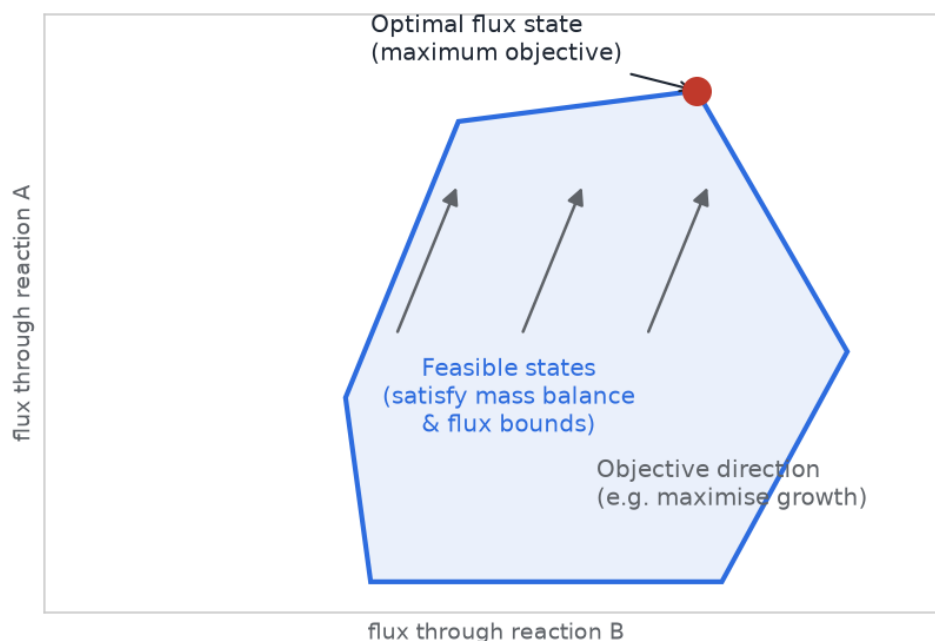


Figure 1.2. FBA searches the feasible flux space (all states satisfying mass balance and bounds) for the point that best serves the objective.

FBA is fast, scales to genome-scale models, and needs no kinetic data, which is exactly why it underpins almost every analysis in this toolbox. But two caveats recur throughout the manual. The optimum objective *value* is unique, yet the flux distribution that achieves it often is **not** (many corners tie): this is why **parsimonious FBA** and **flux variability analysis** exist (Part V). And FBA assumes the cell actually behaves optimally for the chosen objective, which is a modelling choice, not a law of nature.

1.5 What you can and cannot conclude

- **Good for:** maximum theoretical yields and growth rates; which nutrients limit growth; essential genes/reactions; growth-vs-production trade-offs; qualitative re-routing of flux after a knockout or a medium change; ranking engineering targets.
- **Weak or silent on:** absolute metabolite concentrations; enzyme kinetics and rate limitation; regulation and signalling; time courses; anything requiring parameters the reconstruction does not contain.
- **Only as good as the model:** a gap in the reconstruction, a wrong bound, or an unbalanced reaction propagates into every prediction, which is why Part VI on curation comes before you trust a design.

1.6 A map of the toolbox

Everything is reachable from the menu bar (**File, Edit, View, Settings, Tools, Help**) and the central tabs. The table below shows where each capability lives; the rest of the manual works through them in order.

You want to...	Go to	Covered in
Open / save / export a model	File menu	Ch. 2–3
Edit reactions, medium, objective, categories	Edit menu / Tools > Edit Model	Ch. 4

You want to...	Go to	Covered in
Browse reactions, metabolites, genes	Explorer (left dock)	Ch. 3
See the network / flux maps	Network Map tab; Tools > Visualization	Ch. 5
Interactive Escher maps & FBA navigator	Escher Visualizer tab	Ch. 5
Run any analysis	Analysis tab	Ch. 6–12
Design knockouts / amplification targets	Analysis tab (Strain design)	Ch. 9
Add a heterologous pathway	Pathway Design tab	Ch. 10
Build a microbial community	Tools > Consortia modelling	Ch. 12
Compare engineering rounds	Strategy & Escher Visualizers	Ch. 5, 13
Manage downloads / cache / results folder	Settings menu	Ch. 14

PART II

Getting started

Launching the application, finding your way around the window, and loading the first model.

2. Installation and the interface

2.1 Launching the application

GSM ToolBox ships as a self-contained Windows application: no Python installation, no internet connection, and no configuration are required to start. Launch it from the desktop shortcut or by running the bundled `GSM_ToolBox.exe`. The window opens **maximised**. The very first time you use an online feature (fetching a reaction database, looking up an enzyme) the app will reach out to the internet, but every download is cached permanently so later runs work offline.

2.2 A tour of the window

The interface has four regions. On the **left** are two docked panels: the **Explorer** (a searchable list of every reaction, metabolite and gene) on top, and the **Categories** panel (your named groups of reactions) beneath it. The **centre** is a set of tabs. **Analysis**, **Pathway Design**, **Growth Settings**, **Strategy Visualizer**, **Escher Visualizer** and **Network Map**, which is where most work happens. A slim **Information** panel shows details of whatever you have selected; it defaults to the right and can be moved to the bottom from *View*. The **status bar** at the very bottom reports background tasks and their progress.

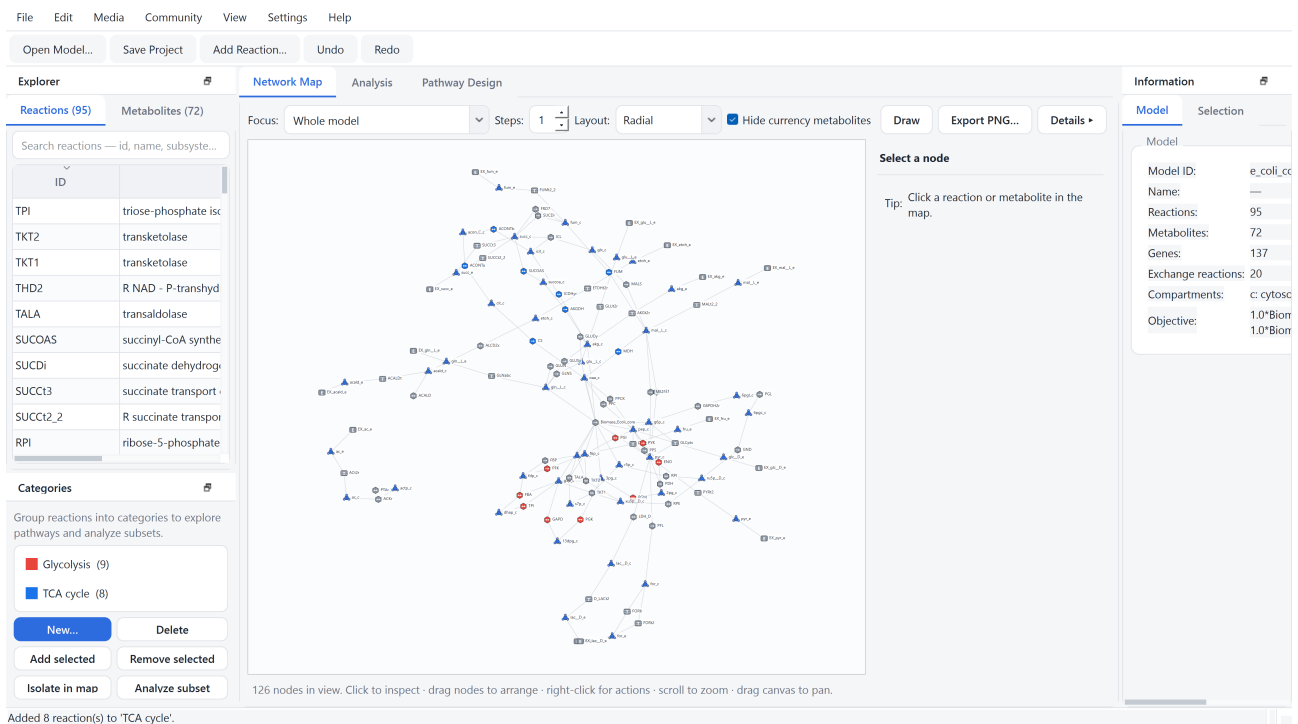


Figure 2.1. The main window: Explorer and Categories (left), the central tabs (here the Network Map), and the Information panel (right).

2.3 The menus

- **File.** Open, save and export models and projects.
- **Edit.** Add/edit/remove reactions, set the objective, undo and redo.
- **View.** Show or hide the docks and choose the Information panel's position.
- **Settings.** Manage downloaded data and the results output folder.
- **Tools.** A single launcher for every utility, grouped as *Edit Model*, *Analysis*, *Pathway Design*, *Consortia modelling* and *Visualization* (including *Escher Mapping*). Everything in the tabs is also reachable here.

- **Help.** About and version information.

Tip. The docks are movable and closable. Drag a panel out to float it, or close it and bring it back from the *View* menu. If the layout ever gets untidy, the *View* menu restores each panel.

PART III

Loading, inspecting and tweaking a model

Opening a model, reading it with the Explorer, and setting the three things that define a simulation: the reactions, the medium and the objective.

3. Loading and inspecting a model

3.1 Opening a model

Use *File > Open Model* to load a model in **SBML** (.xml / .sbml), **JSON**, or **MATLAB** (.mat) format. The three formats the field uses, or *File > Open Example Model* to load the bundled *E. Coli* core model, a small, well-behaved network that is ideal for learning. Large or awkwardly-formatted files (for example the 21 MB BiGG universal model) are loaded through a tolerant reader that repairs common problems automatically, so loading rarely fails.

Once a model is open, the Explorer fills with its contents and the Information panel's **Model** tab summarises the reaction, metabolite and gene counts, the compartments, and the current objective. *View > Model info...* opens the same summary in a larger, scrollable window.

3.2 The Explorer: searching everything at once

The Explorer has tabs for **Reactions**, **Metabolites** and **Genes**. Its search box is deliberately powerful: it matches far more than the visible columns. Typing into it filters on identifiers, names, subsystems, gene rules, the ids and names of the metabolites a reaction touches, and cross-reference annotations. All at once. Multiple words narrow progressively, so *glycolysis pyk* finds pyruvate kinase within glycolysis. Select any row to populate the Information panel; double-click (or use the *Edit* menu) to edit it; right-click for quick actions.

The screenshot displays the GSM Toolbox application window. At the top is a menu bar (File, Edit, Media, Community, View, Settings, Help) and a toolbar with buttons like 'Open Model...', 'Save Project', 'Add Reaction...', 'Undo', and 'Redo'. Below the toolbar is a tabbed interface with 'Explorer', 'Network Map', 'Analysis', and 'Pathway Design'. The 'Explorer' tab is active, showing a table of reactions. The 'Network Map' tab is also visible, showing a complex metabolic network diagram. The 'Information' panel on the right displays details for the selected reaction 'PGI'.

ID	Name
PGM	phosphoglycerate n
PGL	6-phosphogluconol
PGK	phosphoglycerate k
PGI	glucose-6-phosphat
PFL	pyruvate formate ly
PFK	phosphofructokinase
PDH	pyruvate dehydroge
O2t	R o2 - transport-diff
NH4t	R ammonia reversib
NADTRHD	NAD transhydrogen

Categories: Glycolysis (9), TCA cycle (8)

Information panel details for Reaction - PGI:

- Name: glucose-6-phosphate isomerase
- Equation: D-Glucose 6-phosphate <=> D-Fructose 6-phosphate
- Bounds: [-1000, 1000]
- Reversible: yes
- Subsystem: —
- EC number: —
- Gene rule: b4025

Figure 3.1. Selecting a reaction in the Explorer shows its equation, bounds, gene rule and identifiers in the Information panel.

3.3 Reading model quality before you trust it

Before building on a model it is worth a quick sanity check. The **Quality report** (Analysis tab, covered fully in Chapter 8) scores the model out of 100 and lists mass- or charge-imbalanced reactions, reactions missing a gene rule, dead-end metabolites and blocked reactions. A model with many imbalances or a growth rate that looks physically implausible should be curated (Chapter 8) before

you rely on its predictions.

4. Tweaking basic parameters

Three things define what a simulation computes: the **reactions and their bounds** (the network), the **growth medium** (the environment), and the **objective** (what the cell is assumed to be doing). This chapter covers all three, plus adding reactions and saving your work. Every edit is undoable.

4.1 Editing a reaction

Select a reaction and open it from the *Edit* menu (or double-click) to change its **flux bounds** (lower and upper), its **gene–reaction rule**, name or subsystem. The bounds are the single most common tweak: setting a lower bound of zero makes a reaction irreversible; widening or tightening a bound represents relieving or imposing a capacity limit. *Edit > Set Selected as Objective* changes what FBA maximises. *Edit > Undo / Redo* reverses any change.

Caution. Changing a reaction's bounds changes every subsequent simulation. If a result surprises you, check that the bounds and medium are what you intended. An accidental open or closed exchange is the usual culprit.

4.2 Adding a reaction: the reaction builder

Edit > Add Reaction opens an interactive **builder** rather than a bare text field. You add reactant and product rows, each with a stoichiometric coefficient and a **searchable metabolite picker**; choose **reversible (<->)** or **irreversible (->)** (which sets the bounds); and fill in the name, EC number, gene rule and subsystem. A live preview shows the assembled equation, and the **Show reaction** button draws it visually. The 2-D structures of the metabolites (from PubChem/KEGG, cached offline) with each compound's name and coefficient beneath its image.

The picker searches your model's metabolites by name. *Search a metabolite (database / KEGG)* finds compounds in a loaded reaction database or live from KEGG; a chosen compound keeps its real **name**, a metabolite id is suggested (and editable), and its database id is recorded as an annotation. If the compound matches one already in your model (by cross-reference) that existing metabolite is **reused**, so the new reaction connects to the network instead of creating a floating duplicate. *Load reaction by EC number* pastes an EC number and fetches the corresponding reaction from KEGG, pre-filled with named metabolites.

Reaction ID: PYRDC

Name: Pyruvate decarboxylase (example)

Search a metabolite (database / KEGG)... Load reaction by EC number...

Reactants (consumed)

1.0000 Pyruvate (pyr_c)

+ Add metabolite

Products (produced)

1.0000 Acetaldehyde (acald_c)

1.0000 CO2 (co2_c)

+ Add metabolite

☒ Irreversible (→) ☐ Reversible (↔)

Lower: 0.00 Upper: 1000.00

Gene rule (GPR): e.g. b0001 and b0002

Subsystem:

EC number: —

Preview: [Show reaction](#)

PYRDC: 1 pyr_c --> 1 acald_c + 1 co2_c

Pyruvate Acetaldehyde CO2

OK Cancel

Figure 4.1. The reaction builder: metabolite pickers with stoichiometry, a reversibility choice, EC/gene-rule fields and a live preview.

4.3 Setting the growth medium

Tools > Edit Growth Medium (also under the Growth Settings tab) lists every **exchange reaction** with an editable maximum uptake rate. This is how you define the environment: choose the carbon source and its uptake rate, set which ions and nutrients are available, and set a value to zero to remove a nutrient entirely. An aerobic/anaerobic toggle opens or closes the oxygen exchange. Because every simulation runs under the current medium, this is often the first thing to set and the first thing to check when a result looks wrong.

Set the maximum uptake rate (mmol/gDW/h) for each nutrient the cell can import. A value of 0 removes the nutrient from the medium.

☒ Aerobic (oxygen available)

Exchange	Name	Max uptake
EX_ac_e	Acetate exchange	0.000
EX_acald_e	Acetaldehyde exchange	0.000
EX_akg_e	2-Oxoglutarate exchange	0.000
EX_co2_e	CO2 exchange	1000.000
EX_etoh_e	Ethanol exchange	0.000
EX_for_e	Formate exchange	0.000
EX_fru_e	D-Fructose exchange	0.000
EX_fum_e	Fumarate exchange	0.000
EX_glc_D_e	D-Glucose exchange	10.000
EX_gln_L_e	L-Glutamine exchange	0.000

OK Cancel

Figure 4.2. The growth-medium editor controls nutrient uptake and oxygen availability.

Why it matters. Yields are always relative to a substrate and a condition. “Grams of product per gram of glucose, aerobically” is a claim about the medium as much as the strain. Fix the medium deliberately before comparing designs.

4.4 Choosing the optimisation objective

The Analysis tab's **Optimization objective** control (a button at the top that opens the objective editor) steers what simulations maximise: *Maximise growth* (the biomass reaction, detected automatically), *Maximise a product* (pick a target exchange), or *Balance growth & product*. A weighted combination that rewards a target chemical alongside growth. For full control, the **custom / weighted objective** editor lets you add any number of reactions, each with a weight (for example 60% biomass, 40% product). With *Treat weights as relative priorities* enabled, each weight is normalised by that reaction's maximum attainable flux, so the percentages behave intuitively even when the reactions operate at very different magnitudes.

Build an objective from one or more reactions, each with a weight. Example: 0.6 for the biomass reaction and 0.4 for a product exchange to balance growth with production.

Direction: Maximize ▼

Reaction	Weight
Biomass_Ecoli_core ▼	0.600
EX_ac_e ▼	0.400

☒ Treat weights as relative priorities (normalize by each reaction's max flux)

Figure 4.3. The custom objective editor: combine reactions with weights to define complex optimisation targets.

4.5 Saving, exporting and undo

File > Save Project stores the edited model, your categories, saved strategies and settings in a single .gsmtbx file, so a complete engineering session can be reopened or shared. *File > Export Model* writes the current model back out to SBML, JSON or MATLAB for use in other tools. Every model edit is reversible with *Edit > Undo / Redo*.

PART IV

Exploring the network visually

Four complementary ways to see a model: the native Network Map, reaction Categories, the interactive Escher Visualizer, and the Strategy Visualizer for comparing engineering rounds.

5. Seeing the metabolic network

5.1 The Network Map

The **Network Map** draws metabolites as molecule icons and reactions with type-specific icons: a hexagon with a double-headed arrow (<->) for reversible reactions, a single arrow (->) for irreversible ones, a membrane badge marked **T** for transporters, and a badge marked **E** for exchange/boundary reactions. Because a whole genome-scale network is an unreadable “hairball”, the map is focused by default. Use the **Focus** box to centre it on a reaction, a metabolite, a category or the whole model; **Steps** controls how many reaction hops are expanded outward; and **Layout** switches between radial, layered (pathway-flow) and force-directed arrangements. Ubiquitous currency metabolites (ATP, water, protons...) are hidden by default to keep the picture legible.

You can **drag nodes** to rearrange the diagram (edges follow), drag the canvas to pan, and scroll to zoom; **Export PNG** saves a high-resolution image. Click a node to select it (highlighting it, selecting it in the Explorer, and filling the Details panel with a metabolite's formula/charge/structure or a reaction's equation/direction/EC/flux). Right-click for actions. After a simulation the edges are coloured and scaled by flux, so you can literally see where carbon flows.

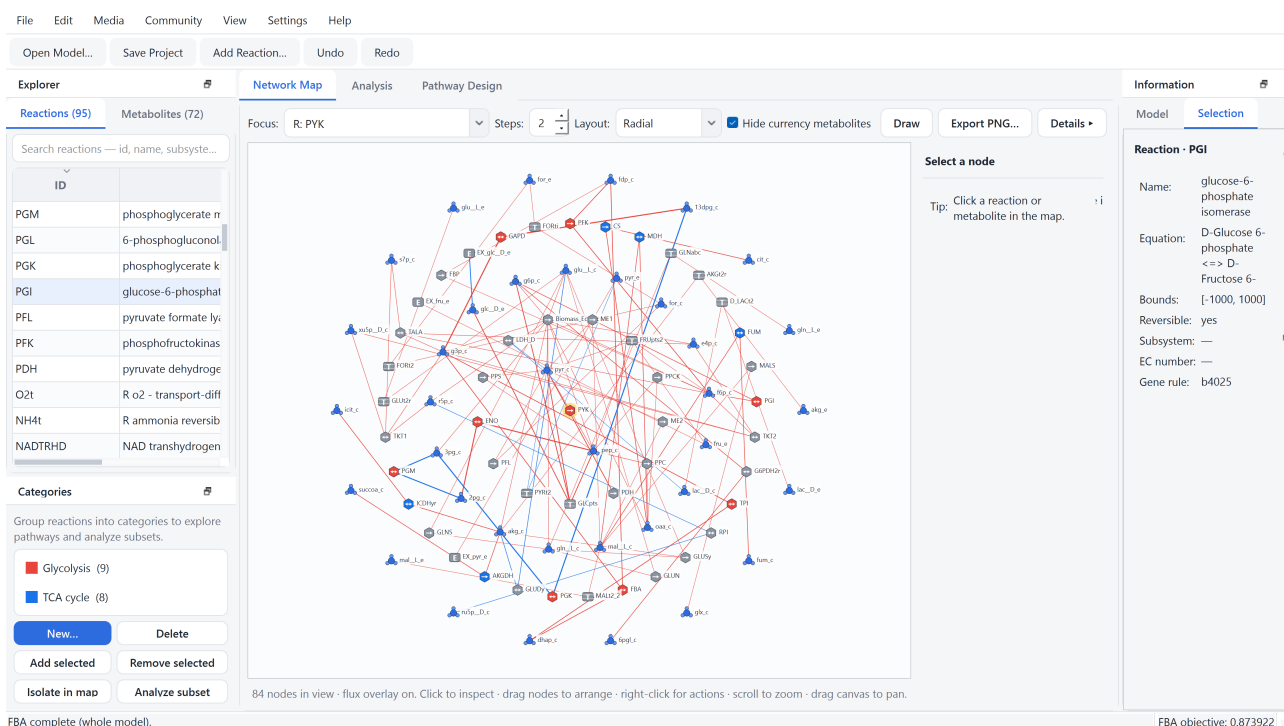


Figure 5.1. A focused neighbourhood with category colours and an FBA flux overlay (edge width and colour encode flux magnitude and direction).

5.2 What the colours mean

Colour carries two independent facts, and they are kept strictly apart. **Green is reserved, in every view, for metabolites that are not native to the model.** The compounds a designed pathway introduces. That is the distinction you act on, so no compartment is allowed to borrow the hue. Every other colour is a compartment, and the legend above the map lists them **for the model currently open**: a three-compartment model shows three entries, and a compartment that carries no metabolites is not listed at all.

5.3 Categories: grouping and isolating pathways

Categories are named, coloured groups of reactions. Typically a pathway you care about. Select reactions in the Explorer and click *Add selected* in the Categories panel (or right-click in the map). *Isolate in map* shows only that category; *Analyze subset* runs any analysis restricted to it. For a subset analysis the toolbox builds a standalone sub-model in which every connection to the rest of the network becomes a free input/output exchange, so a pathway can be studied on its own. Categories are also the unit of “scope” used throughout the Analysis tab and in the Escher Visualizer.

5.4 The Escher Visualizer: interactive maps and an FBA navigator

The **Escher Visualizer** tab (also opened from *Tools > Visualization > Escher Mapping*) embeds the field-standard **Escher** renderer for publication-quality, interactive flux maps. Unlike curated Escher maps, which exist for only a handful of well-known models. The toolbox **generates a map automatically for any model you load**, so it works on your own reconstructions. Use the **Focus** control to draw the whole model or a single category (recommended for large models), then choose one of three modes:

- **Single strategy.** Draw one saved strategy's flux magnitude on the map (a sequential colour scale; arrow width grows with |flux|).
- **Difference (A-B).** The change in flux between two saved strategies on a diverging red/blue scale, so you can see exactly which reactions carry more or less flux after an intervention. This is the single most useful view for engineering.
- **FBA navigator (live).** An interactive mode inspired by Escher-FBA: click any reaction on the map, tighten or open its bounds or knock it out, and the toolbox re-solves FBA instantly and re-colours the map, so you can watch flux re-route in real time. The objective value updates with each edit, and *Reset all bounds* returns to the original model.

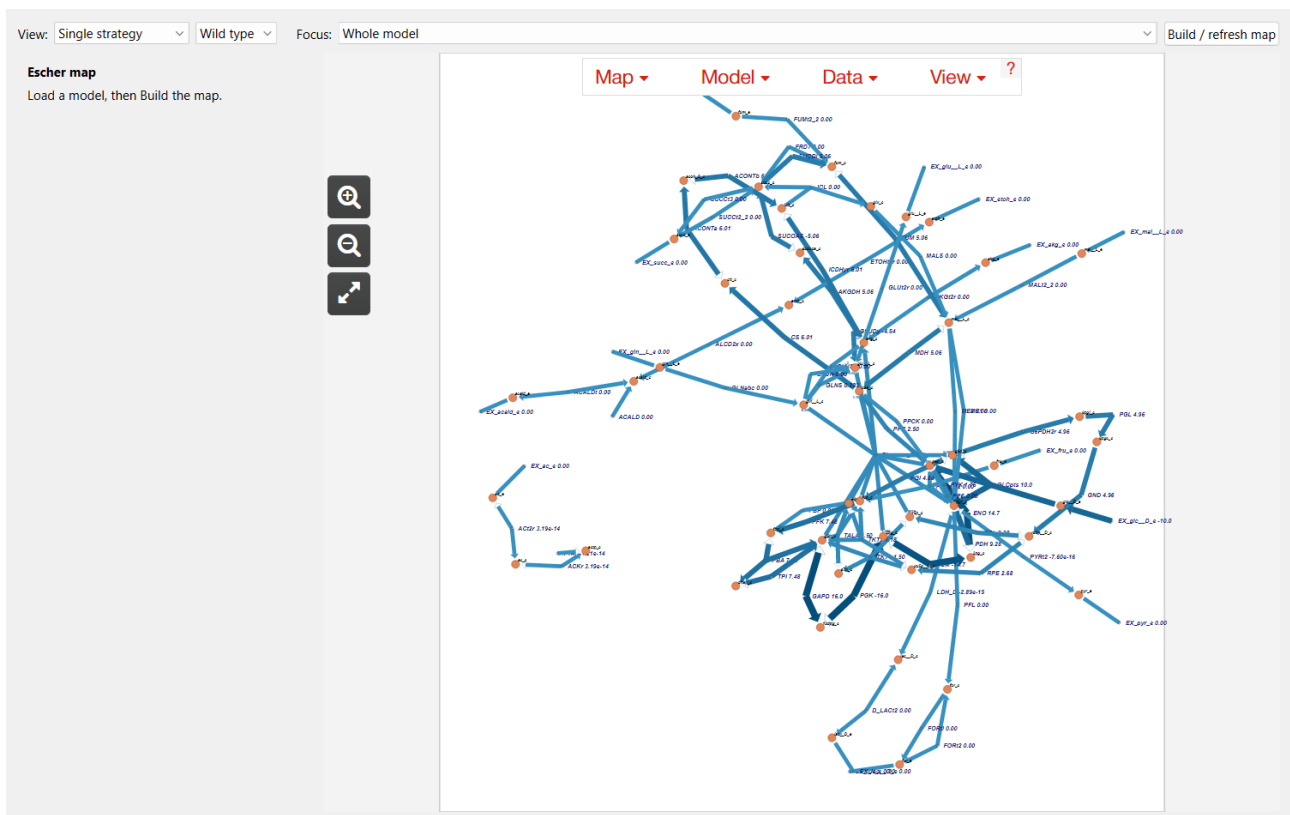


Figure 5.2. The Escher Visualizer: an auto-generated interactive Escher map of the model with an FBA flux overlay (arrow colour and width encode flux; each reaction is labelled with its flux value). The mode, strategy and focus controls sit along the top.

Why it matters. The difference map answers “what did my knockout actually change?” at a glance, and the live FBA navigator turns “what if I constrain this reaction?” into a one-click experiment. the fastest way to build intuition about how a network responds.

5.5 The Strategy Visualizer: comparing engineering rounds

A **strategy** is a named, solved flux state. The model after a defined set of edits, with FBA/pFBA solved on it. Saving a strategy after each engineering round (a knockout, an added pathway, a medium change) lets you compare rounds quantitatively in the **Strategy Visualizer**: a single flux map, a **difference map** between any two rounds, a reactions×strategies **heatmap**, a **titre waterfall** (product flux after each round), and a **parallel-coordinates** plot of flux trajectories. Strategies are saved inside the .gsmtbx project. The worked example in Chapter 13 uses these views to tell the story of a design.

PART V

Core simulations

The everyday analyses: predicting a flux state (FBA/pFBA), measuring its flexibility (FVA), reading what limits it (shadow prices), and simulating mutants (MOMA/ROOM).

6. Core flux simulations

Every analysis is launched from the **Analysis** tab and runs in a background process, so the interface stays responsive. Each opens a **settings dialog** first (always including tick-boxes to exclude transport and exchange reactions from the results, since they are rarely the engineering target), runs on the whole model or a chosen category (the **Scope** selector at the top), and returns a sortable, exportable table and, where relevant, a plot. A **Save Results** button writes tables as CSV and plots as PNG+PDF to your results folder. In any result table you can **right-click a row** that names a reaction to show it in the map, view its information, set it as the objective, or add it to a category.

6.1 Flux Balance Analysis (FBA)

Purpose. Predict a steady-state flux through every reaction that maximises the objective, given the medium and bounds. The baseline prediction of growth rate and metabolic activity.

How to run it. Confirm the objective (e.g. *Maximise growth*) and click *Flux Balance Analysis (FBA)*. The objective value (the predicted growth rate in h^{-1} , if biomass) appears in the status bar; the full flux vector fills the results table; and the Network Map colours active reactions. **How to read it.** A growth rate near the experimentally-known value is a good sanity check. Scan the largest-magnitude fluxes to see which pathways dominate; look at the exchange fluxes to see what is taken up and secreted.

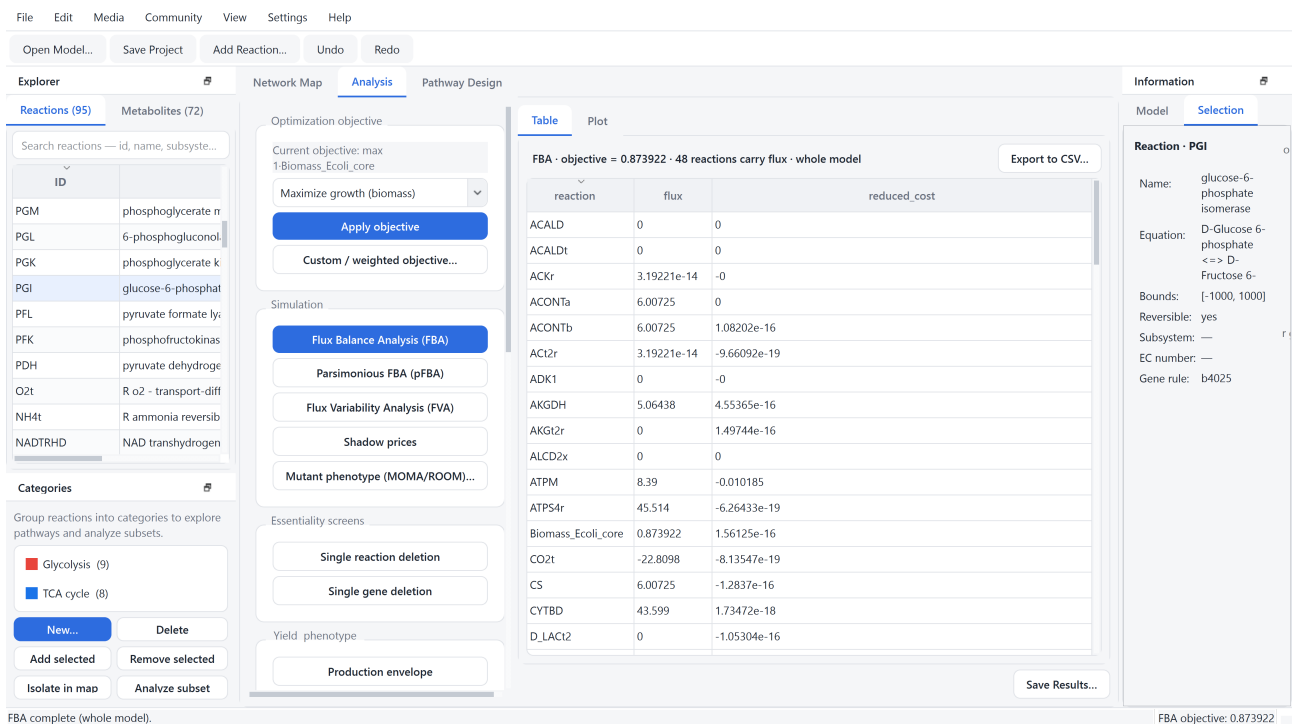


Figure 6.1. FBA results: the objective control, the method buttons and the per-reaction flux table.

6.2 Parsimonious FBA (pFBA)

Purpose. Among all flux distributions that achieve the optimal objective, find the one that uses the **least total flux**. A cleaner, more biologically realistic prediction, reflecting selection for minimal enzyme investment [2]. **When to prefer it.** Use pFBA instead of plain FBA whenever you want a single representative flux state (for a map, a strategy, or a reference for mutant simulation): it removes the arbitrary internal cycles that plain FBA can leave in.

6.3 Flux Variability Analysis (FVA)

Purpose. For each reaction, compute the **minimum and maximum flux** it can carry while the objective stays at (or near) its optimum [3]. This exposes which parts of metabolism are rigidly determined and which are flexible. **Key setting.** The *fraction of optimum* (e.g. 1.0 for the strict optimum, 0.9 to allow near-optimal states) controls how much growth you are willing to trade for flexibility. **How to read it.** A reaction whose min and max are both zero is **blocked**; one whose range is wide is a candidate for regulation or a branch point; one pinned to a single value is essential to the optimal state.

Rule of thumb. Run FBA first to confirm the optimum, then FVA at fraction = 0.9–0.95 to see which reactions still have room to move. Wide-ranging reactions near your product are the ones worth controlling.

6.4 Shadow prices and reduced costs

Purpose. Read, straight from the FBA solution, **what limits the objective**. A metabolite's **shadow price** is how much the objective would change if one more unit of that metabolite were made available. Large values flag the true bottlenecks. A reaction's **reduced cost** is how much the objective would change per unit flux if an inactive reaction were forced on. Together they point to what to feed, transport, or overproduce. These dual values are reported automatically with the FBA solution [1].

6.5 Mutant simulation (MOMA / ROOM)

Purpose. Predict a knockout mutant's flux state more realistically than FBA, which assumes the mutant instantly re-optimises for growth. Immediately after a knockout a cell is closer to its old (wild-type) flux pattern. **MOMA** finds the mutant distribution closest to wild-type [19]; **ROOM** minimises the number of reactions whose flux changes appreciably [20]. **How to run it.** Select the reactions to knock out in the Explorer, then run *Mutant phenotype (MOMA/ROOM)*; the wild-type reference is computed by pFBA. Use these when predicting the immediate phenotype of a fresh knockout rather than an evolved strain.

PART VI

Phenotypes, yields and model quality

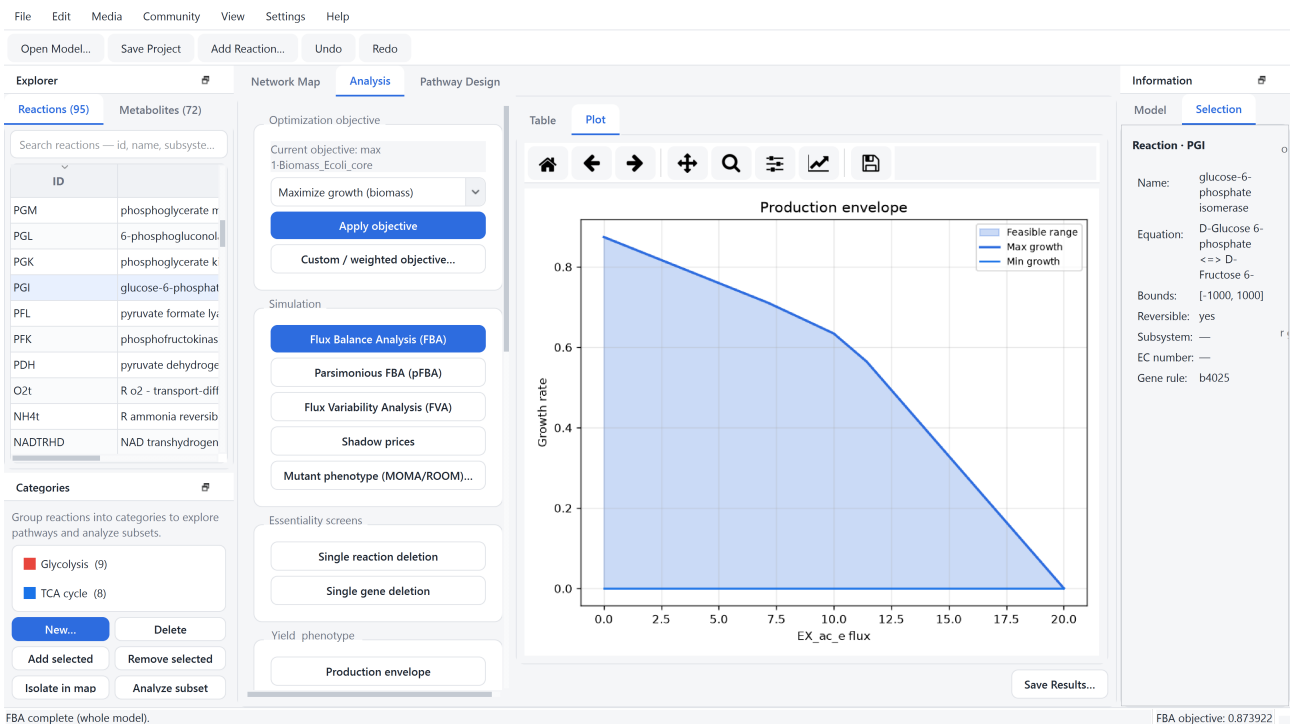
Mapping trade-offs and operating windows (envelopes, phase planes, robustness, sampling), and making sure the model is sound before you trust a design (essentiality, quality report, gap-filling).

7. Phenotype and yield analyses

7.1 Production envelope

Purpose. Draw the **growth-versus-production trade-off**. The single most important picture in strain design. You choose a target product exchange; the toolbox scans the product flux and, at each level, computes the minimum and maximum feasible growth, tracing the boundary of the yield space [5].

How to read it. A line that slopes down to the right means growth and production compete (making product costs growth). A non-zero *lower* edge at high growth means production is **growth-coupled**. The cell cannot grow fast without also making the product, which is exactly what knockout strain design tries to achieve (Chapter 9).



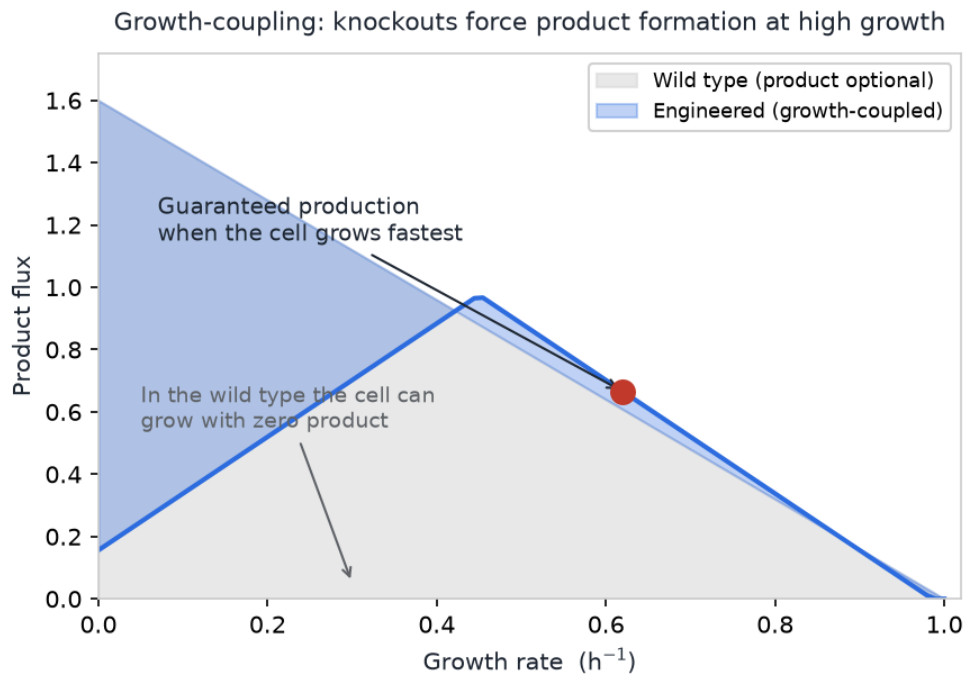


Figure 7.2. The engineering goal: knockouts lift the lower edge of the envelope so that fast-growing cells are forced to secrete the product.

7.2 Robustness analysis

Purpose. Test how sensitive the objective is to forcing a single reaction to a range of fixed values. Revealing tipping points where the phenotype changes abruptly. **How it works.** The control reaction is fixed at successive values and the objective re-optimised at each; the default scan range is taken from an FVA of the control reaction, and infeasible points are dropped. Useful for finding the flux at which growth collapses, or the level of a by-product the cell will tolerate.

7.3 Phenotypic phase plane (PhPP)

Purpose. Map how the objective responds to **two** environmental variables at once. Classically carbon versus oxygen uptake. Exposing distinct metabolic regimes (respiration, fermentation, mixed) separated by sharp lines of constant shadow-price ratio. The objective is optimised over a 2-D grid of the two fixed fluxes and rendered as a heatmap [5]. Reading the boundaries tells you how the strain switches strategy as its environment changes.

7.4 Flux sampling (distributions)

Purpose. Instead of a single optimal state, characterise the **whole feasible space**. The toolbox samples many valid flux distributions and shows a per-reaction distribution (violin/spread), conveying uncertainty and the presence of alternative optima. Where FBA gives one answer and FVA gives a range, sampling gives the shape of the range. Useful when you want to know how *likely* a reaction is to carry flux, not just whether it can.

8. Essentiality and model curation

Curation is not optional housekeeping. A design is only as trustworthy as the model it is built on. This chapter's tools both find engineering targets (essential reactions to avoid knocking out) and catch the errors that would otherwise corrupt every downstream prediction.

8.1 Gene and reaction deletion (essentiality)

Purpose. Find genes or reactions whose removal abolishes or strongly reduces growth. **Single reaction deletion** and **single gene deletion** knock out each element in turn and report the resulting growth and the growth ratio to wild-type; a ratio near zero marks an **essential** element [4]. **Why both matter for engineering.** Essential reactions are the ones you must *not* knock out in a design; conversely, the non-essential set is your search space for interventions. The result is a heatmap-ready table you can sort by impact.

8.2 The quality report

Purpose. Assess the model's structural soundness. The built-in checks count mass- and charge-imbalanced reactions, reactions lacking a gene rule, dead-end metabolites and blocked reactions, and combine them into a 0–100 score in the spirit of the MEMOTE standard [13]. **How to act on it.** Mass/charge imbalances distort yields and should be fixed; dead-ends and blocked reactions often reveal gaps that gap-filling can close (below). A separate line counts reactions whose balance *cannot be checked* because a participant carries no chemical formula. That is a gap in the database, not a fault in the reaction. It is reported apart from the imbalances and does not lower the score. The reaction editor can fetch the missing formula, after which the reaction gets a real verdict.

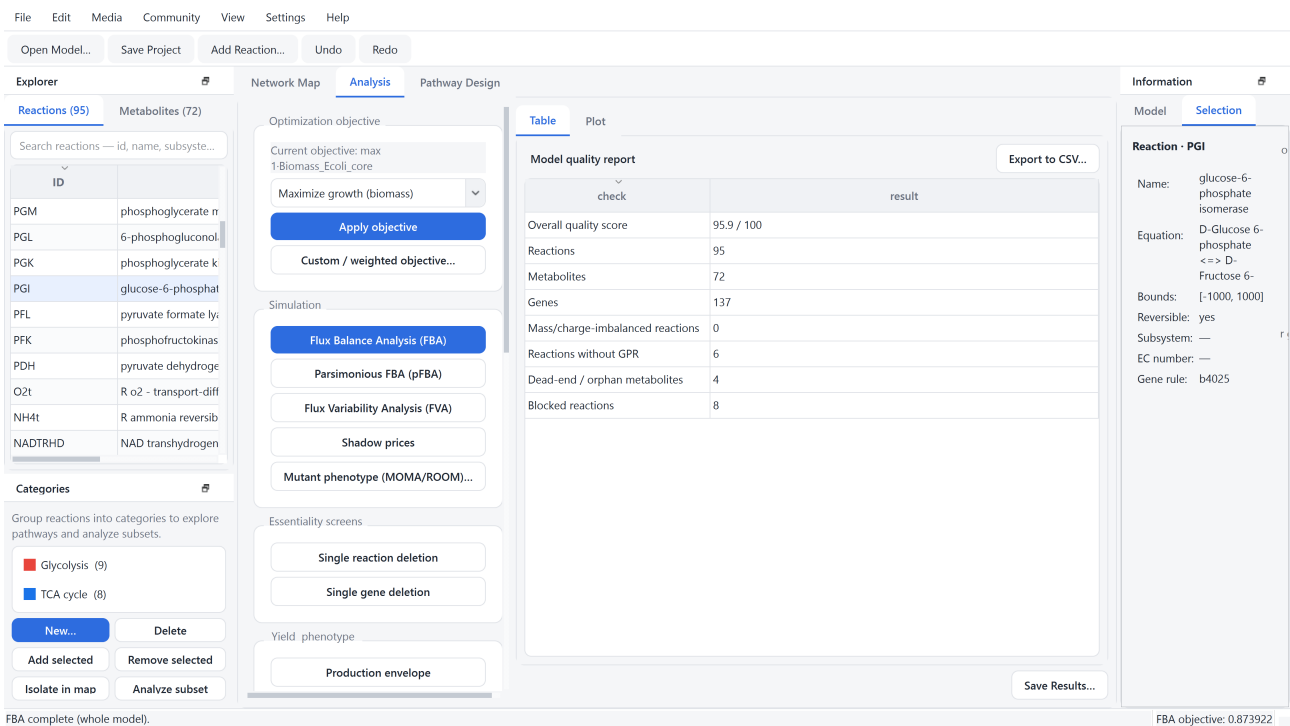


Figure 8.1. The model quality report: an overall score and per-issue lists.

8.3 Blocked reactions

Purpose. List reactions that cannot carry any flux under the current constraints. Usually signs of a network gap (a missing neighbouring reaction) or an overly tight bound. Detected via FVA (a feasible range of exactly zero) [3]. A large blocked set in the pathway you care about is a warning that the model cannot yet represent it.

8.4 Gap-filling

Purpose. Suggest the smallest set of reactions, drawn from a user-supplied **universal model**, that would let the model achieve something it currently cannot, either *grow* on a defined medium, or

produce a chosen metabolite [12]. **Gap-fill for growth** repairs a model that will not grow; **gap-fill for a metabolite** finds what is missing to make a compound. This is closely related to heterologous pathway design (Chapter 10), which uses the same idea to add non-native chemistry deliberately.

PART VII

Designing strains for higher yield

Turning analysis into intervention: which reactions to knock out, amplify or down-regulate so the host makes more of your product.

9. Strain design

9.1 The design problem, in one picture

Wild-type cells make just enough of most metabolites for growth and secrete a product only if it happens to be favourable. The engineering goal is to change the network so that **making the product is unavoidable when the cell grows**. “growth-coupling.” Then selection for fast growth also selects for high production, and the strain stays productive over long fermentations. Figure 7.2 showed the target: lift the lower edge of the production envelope. The tools below find the interventions that do it.

9.2 Knockout strain design (OptKnock / RobustKnock / OptCouple / Heuristic)

Knockout strain design opens one dialog where you choose the **method**, the **solver** (Auto / SCIP / GLPK, or a commercial solver if installed), the target product and biomass reactions, the maximum number of knockouts, and the **minimum growth** (as a fraction of wild-type). Each solution lists the knockouts and the predicted growth and product flux. The methods differ in what they guarantee:

- **OptKnock**. Maximises product synthesis at the cell's growth-optimal state, over a limited number of knockouts (a bi-level mixed-integer program) [7].
- **RobustKnock**. Like OptKnock but maximises the *guaranteed minimum* production, accounting for competing alternative-optimal flux states [8]. Safer when you need a production floor.
- **OptCouple**. Explicitly maximises the strength of growth-coupling [10].
- **Heuristic (evolutionary)**. A fast metaheuristic search for when the exact mixed-integer methods are intractable or return nothing.

Tip. If a run finds no design, loosen the constraints: lower the minimum-growth fraction (e.g. to 0.01), allow more knockouts, choose the “Any/Diverse” approach, or switch to the Heuristic method. Exchange/transport/biomass/maintenance reactions are protected from knockout by default so only meaningful interventions are proposed.

9.3 FSEOF: amplification and knock-down targets

Purpose. Nominate **over-expression** and **down-regulation** targets quickly, without mixed-integer optimisation. **Flux Scanning based on Enforced Objective Flux (FSEOF)** steps the enforced product flux upward while re-optimising growth; reactions whose flux *rises* in step are amplification targets, and those whose flux *falls* are knock-down targets [9]. It is a fast first pass that pairs well with knockout design: use FSEOF to shortlist, then OptKnock/RobustKnock to commit.

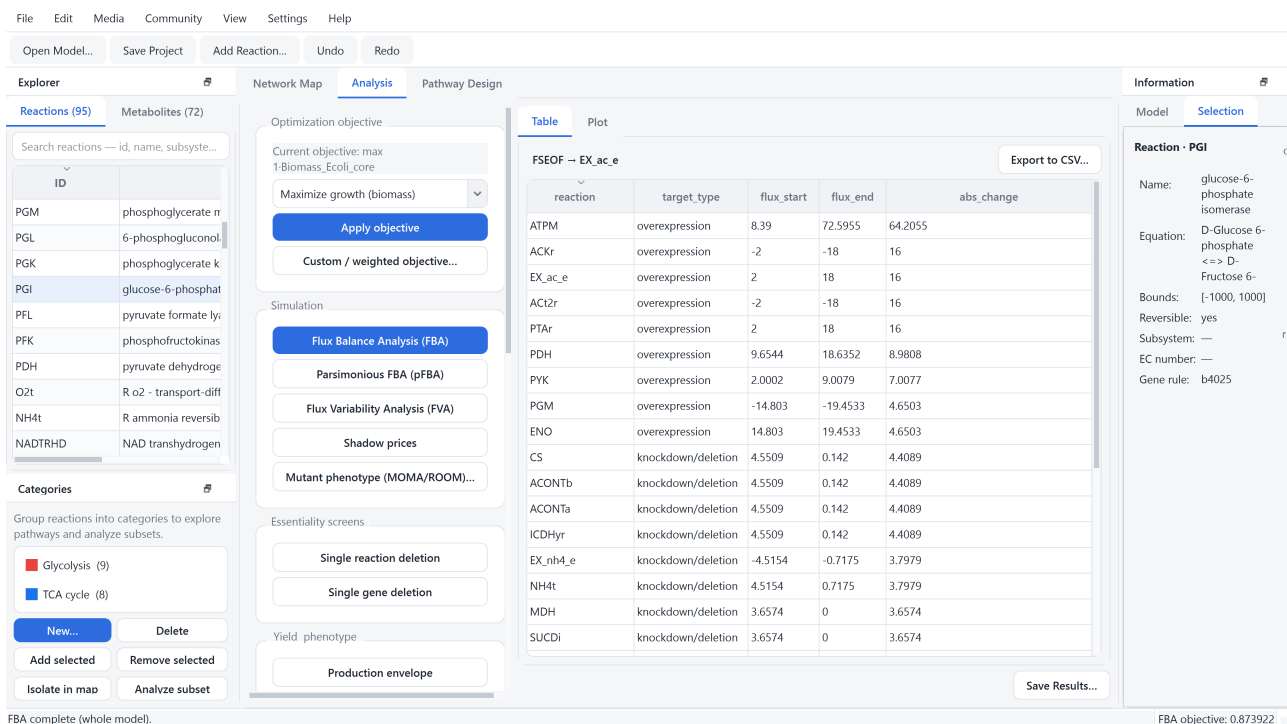


Figure 9.1. FSEOF results: reactions ranked by how their flux changes as product formation is enforced (amplify vs knock-down).

9.4 Overproduce a metabolite

Purpose. Target accumulation of a **metabolite** rather than flux through a particular reaction. Usually the real goal. Search for the target metabolite; the toolbox adds a demand that forces its net production (so it works even when the metabolite is a reactant, or is made by reversible reactions) and runs an FSEOF scan to nominate which reactions to amplify or knock down. The result table includes a **direction** column showing which way each reaction must run. Use this when you know the chemical you want but not the exact reaction that should carry the flux.

9.5 Reading and prioritising designs

- Prefer designs with a **non-zero guaranteed product flux** at high growth (check with a fresh production envelope on the modified model).
- Fewer knockouts are easier to build. Weigh predicted gain against genetic effort.
- Cross-check nominated knockouts against the **essentiality** results (Chapter 8) so you never propose removing an essential reaction.
- Save each candidate as a **strategy** and compare them in the Strategy/Escher Explorers before committing (Chapters 5 and 13).

PART VIII

Heterologous pathway design

When the host cannot make your target at all: importing new chemistry from a universal reaction database, and finding the genes to realise it.

10. Designing heterologous pathways

10.1 The idea

Many valuable chemicals are not native to industrial hosts. **Pathway design** finds a set of reactions. Drawn from the union of known biochemistry, that, added to your host, lets it convert its own metabolites into the target. It is the computational first step of building a producing strain for a non-native compound; once a route is added, every other tool in this manual applies to the engineered host.

10.2 Reaction databases and the offline cache

Open the **Pathway Design** tab. On first use the toolbox offers to download two **recommended databases**. The BiGG universal model and the MetaNetX universal, stored permanently and loaded automatically thereafter. You can add more with *Load reaction database* (a local file) or *Fetch online*, and keep several loaded at once. The sources differ in coverage:

- **BiGG universal**. Shares identifiers with BiGG-based hosts; the safest match for standard *E. Coli*/yeast models.
- **MetaNetX universal**. A reconciliation of KEGG, MetaCyc, BiGG, SEED and Rhea (~33,000 mass-balanced reactions with EC numbers); the broadest coverage of exotic products.
- **ModelSEED** biochemistry. SEED namespace, with compound names and EC numbers.
- **KEGG (focused)**. Built live around a product you name: the toolbox resolves the name to a compound and fetches the reactions and EC numbers around it.

Manage reaction databases lets you tick which loaded databases are included in the search and permanently delete a downloaded one. Every download and every built database is cached under `~/.gsm_toolbox/databases`, reused instantly on later runs, and manageable from *Settings* > *Manage Data* (Chapter 14).

10.3 Different databases, different identifiers. Handled automatically

The same compound is written differently in each resource (glucose is `glc__D` in BiGG, `C00031` in KEGG, `MNXM41` in MetaNetX). GSM ToolBox matches a database to your model *across* identifier systems by comparing every cross-reference each metabolite carries, so a KEGG or MetaNetX database connects to a BiGG host even though their ids differ. Shared metabolites are unified before the search, and matched reactions are translated into your model's namespace when you apply the pathway. including collapsing foreign compartments into the host's, so applying a route never creates phantom organelles.

Note. For the broadest matching, accept the one-time download of the MetaNetX compound reference data when prompted. With it, MetaNetX metabolites show real chemical names and carry the full set of cross-references, so they match even models that record no MetaNetX ids.

10.4 Predicting a pathway

Pick the **target metabolite** (the list shows each compound's name and id), set the minimum production flux, and click *Predict pathway*. The route is found by fast **network expansion**: starting from the metabolites the host already makes, the search repeatedly fires database reactions whose substrates are all available until the target becomes reachable, then back-tracks to collect the

precursor-first reaction list. This returns in seconds even against a genome-scale universal database, where a full mixed-integer gap-fill would be intractable.

Clicking *Predict pathway* first opens a **settings window**. By default the search starts from every metabolite the model makes and returns the shortest route. Untick *Use all metabolites/reactions* to choose specific **starting metabolites** (so the route must originate from chosen precursors); supply **preferred EC classes** (e.g. 4.2.3, 2.5.1) to bias the chemistry; cap the **maximum pathway length**; and request several **alternative pathways**. Each is found, then excluded, and the search repeats to yield genuinely different routes.

Predict a heterologous pathway to **Isoprene (isoprene_c)**. By default the search starts from everything your model already makes and finds the shortest route. Adjust the options below for finer control.

Starting metabolites

☒ Use all metabolites/reactions in the model (default)

2-Oxoglutarate (akg_c) ▼ Add →

Remove selected

Search options

Minimum target flux: 0.100 ▲▼

Preferred EC classes: 4.2.3, 2.5.1

Maximum pathway length: 12 ▲▼

Number of pathways to return: 3 ▲▼

Predict Cancel

Figure 10.1. Pathway-design settings: starting metabolites, preferred EC classes, maximum length, and the number of alternative routes.

10.5 Inspecting, drawing and applying a route

When more than one pathway is found, a selector steps through them (each labelled with its reaction count and predicted flux). The predicted reactions show **readable names** (e.g. “L-phenylalanine ammonia-lyase”), a suggested short id (editable), and equations written with compound names. A row of actions sits below the list: *Add pathway to model* adds the reactions (a summary dialog first reports the new reactions and metabolites, and offers to place them in a category and add a secretion route for the product) so you can immediately run FBA and strain design on the engineered host [18]; *Display reaction network* draws the new reactions with the native metabolites they connect to; *Explore alternative pathways* searches for a different route; *Draw pathway scheme* renders a publication-style diagram with 2-D structures; and *Save Results* writes the reactions to CSV.

Right-click any predicted reaction for its full information.

File Edit Media Community View Settings Help

Open Model... Save Project Add Reaction... Undo Redo

Explorer Reactions (95) Metabolites (72)

Search reactions — id, name, subsystem...

ID

PGM	phosphoglycerate n
PGL	6-phosphogluconol
PGK	phosphoglycerate k
PGI	glucose-6-phosphat
PFL	pyruvate formate ly
PFK	phosphofructokinase
PDH	pyruvate dehydroge
O2t	R o2 - transport-diff
NH4t	R ammonia reversib
NADTRHD	NAD transhydrogen

Categories

Group reactions into categories to explore pathways and analyze subsets.

☒ Glycolysis (9)

☒ TCA cycle (8)

New... Delete

Add selected Remove selected

Isolate in map Analyze subset

Network Map Analysis Pathway Design

Design a heterologous pathway: load or fetch reaction databases, choose a target metabolite, and predict the reactions that let the host produce it. Metabolites are matched to your model across identifier systems (BIGG, KEGG, MetaNetX...). Fetched reactions carry EC numbers — right-click a result for details, or use "Find enzymes in UniProt" in the Selection panel to find a gene to express.

Manage reaction databases... Load reaction database... Fetch online...

Target metabolite: Isoprene (isoprene_c) Min flux: 0.100 Predict pathway

Target isoprene_c: 2 heterologous reaction(s) needed. Predicted production flux: 2.41.

Heterologous reactions to add for isoprene_c

reaction	name	EC	equation
DMAPPS	dimethylallyl-PP synthase	2.5.1.1	ipdp_c + dmpp_c --> grdp_c
ISPS	isoprene synthase	4.2.3.27	dmpp_c --> isoprene_c + ppi_c

Export to CSV...

Loaded databases

☒ MetaNetX universal (95 rxns)

1 of 1 database(s) included in the search - 72 searchable metabolite(s).

Information

Model Selection

Reaction - PGI

Name: glucose-6-phosphate isomerase

Equation: D-Glucose 6-phosphate <=> D-Fructose 6-phosphate

Bounds: [-1000, 1000]

Reversible: yes

Subsystem: —

EC number: —

Gene rule: b4025

Loaded MetaNetX universal (95 reactions). FBA objective: 0.873922

Figure 10.2. Pathway Design: a predicted heterologous route with EC numbers, drawn from an online reaction database.

10.6 From an EC number to a gene

A predicted reaction tells you the *chemistry*; to build the strain you need a *gene*. Every reaction carries its **EC number**, shown in the Explorer and the Selection panel. Select a reaction and click *Find enzymes in UniProt* to list reviewed enzymes and their source organisms for that EC number. Concrete candidate genes to express. For example, EC 4.2.3.27 (isoprene synthase) returns the poplar and kudzu enzymes used in published isoprene strains.

Reviewed UniProt enzymes for EC 4.2.3.27 (reaction ISPS).

Candidate genes to express for this step.

UniProt enzymes — EC 4.2.3.27

Export to CSV...

Accession	Protein	Organism	Length	Re
Q50L36	Isoprene synthase, chloroplastic (PalspS)	Populus alba (White poplar)	595	revi
Q9AR86	Isoprene synthase, chloroplastic (PclSPS)	Populus canescens (Grey poplar)	595	revi
Q6EJ97	Isoprene synthase, chloroplastic	Pueraria montana var. lobata (Kudzu vine)	608	revi

Close

Figure 10.3. UniProt enzyme lookup for a reaction's EC number. Candidate genes and their source organisms.

Caution. A predicted pathway is a hypothesis about what is *stoichiometrically* possible. It does not guarantee that a suitable enzyme exists, is soluble in your host, or carries flux in vivo. Treat the UniProt candidates and the route as a starting point for experimental design, not a finished answer.

PART IX

Omics, energy and communities

Making predictions condition-specific with expression data, probing energetic burden, and modelling more than one organism at a time.

11. Omics integration and energy analysis

By default FBA knows only stoichiometry and the medium. Real cells also switch genes on and off. The methods here fold measured gene expression into the model to give **context-specific** predictions, and probe how much energetic burden a strain can bear.

11.1 Preparing an omics dataset

eFlux and GIMME need a two-column table of (**gene id**, **value**) whose ids are in your model's namespace, but public omics data almost never arrives that way. *Prepare omics dataset* (Analysis > Omics & energy) turns a raw table into that form. Choose the file (CSV, TSV or Excel. The delimiter is detected automatically and comment lines are skipped), the **data type** (*transcriptomics* or *proteomics*, mapped to genes; *metabolomics*, mapped to metabolites), the **identifier column** and the **value column(s)**. tick several replicate/sample columns and choose how to combine them (mean, median, max...). Click **Map to model** and the tool matches the data's identifiers to the model across schemes (locus tags, UniProt accessions, gene names, KEGG/ChEBI ids for metabolites. Even several ids packed in one cell, or FASTA headers), then shows a **coverage summary**: how many rows matched, which model genes were covered, and examples of unmatched ids. You can save the prepared two-column table, or load it straight into the model.

Once a dataset is prepared or loaded, an **Omics** tab appears in the main window (it is present only when omics data has been mapped) listing each dataset, its coverage summary and the per-id values that feed the analyses below.

Rule of thumb. The usual reason for low coverage is an identifier-scheme mismatch (gene symbols vs locus tags vs UniProt). Check the coverage summary; if it is low, pick a different identifier column or a data source whose ids match your model.

11.2 eFlux

Purpose. Produce a flux prediction that reflects measured expression. Each reaction's flux capacity is scaled by the expression of its catalysing genes (combined through the gene–reaction rule), so highly-expressed reactions may carry more flux and lowly-expressed ones are throttled [14]. The result is a single condition-specific flux state. Useful for comparing what the cell actually does across conditions.

11.3 GIMME

Purpose. Build a flux distribution consistent with expression while still achieving a required function (e.g. growth). Flux through reactions whose genes fall below an **expression threshold** is minimised subject to an objective-fraction constraint, and an **inconsistency score** quantifies the remaining conflict with the data [15]. Use it when you want a context-specific model that still meets a physiological objective.

11.4 ATP-maintenance sensitivity

Purpose. Measure how much **non-growth energetic burden** a strain can tolerate before growth collapses. Important for stressed or heavily-engineered strains, where product synthesis, protein expression and stress responses all draw ATP. The ATP-maintenance reaction's demand is scanned and growth re-optimised at each level [1], plotting the growth-versus-burden curve.

12. Communities and pathway architecture

12.1 Community (multi-organism) modelling

Purpose. Predict how several organisms grow together and exchange metabolites , for microbial consortia and division-of-labour designs, from *Tools > Consortia modelling > Build Community Model*, member models are merged with organism-tagged intracellular compartments and a shared extracellular pool; community FBA then reports each member's growth [17]. Use *Community member growth* in the Analysis tab to run it. Consortia can achieve conversions no single strain can, by splitting a pathway across specialists.

Dominance and no starvation. A naive community FBA maximises the *summed* biomass, which lets the faster grower take all the shared nutrients and starve the others. The **Consortia objective** settings (a button in the objective panel when a community model is loaded, and offered when you run *Community member growth*) fix this with two controls, following standard community-modelling practice: a **dominance weight** per organism (its relative abundance in the community objective , higher means it dominates) and a **minimum growth per member** (each organism must grow at least this fraction of the maximum it could reach in the community, so no member is starved). If the community becomes infeasible, the minimum-growth requirement is too high for the shared nutrients. Lower it. The result table reports each member's growth, its weight and its growth floor.

12.2 Elementary flux modes (EFMs)

Purpose. Enumerate the minimal, indivisible steady-state pathways through a subnetwork. The fundamental routes from substrates to products, useful for understanding pathway architecture and theoretical yields. Reversible reactions are split so the feasible space is a pointed flux cone; its extreme rays (found by the double-description method) are the EFMs [16]. **Caution:** enumeration is exponential, so EFMs are restricted to a small **category** (a few dozen reactions), not the whole model. Define the subnetwork as a category first, then run *Elementary flux modes* scoped to it.

PART X

Putting it together, and reference

A complete worked example that walks from a fresh model to an improved strain, followed by the operational details, comparison table, glossary, FAQ and references.

13. A complete yield-improvement workflow

This chapter ties the whole manual together with a realistic sequence. The exact numbers depend on your model and product, but the **order of operations** is general. Figure 13.1 summarises it.

A yield-improvement workflow in GSM ToolBox

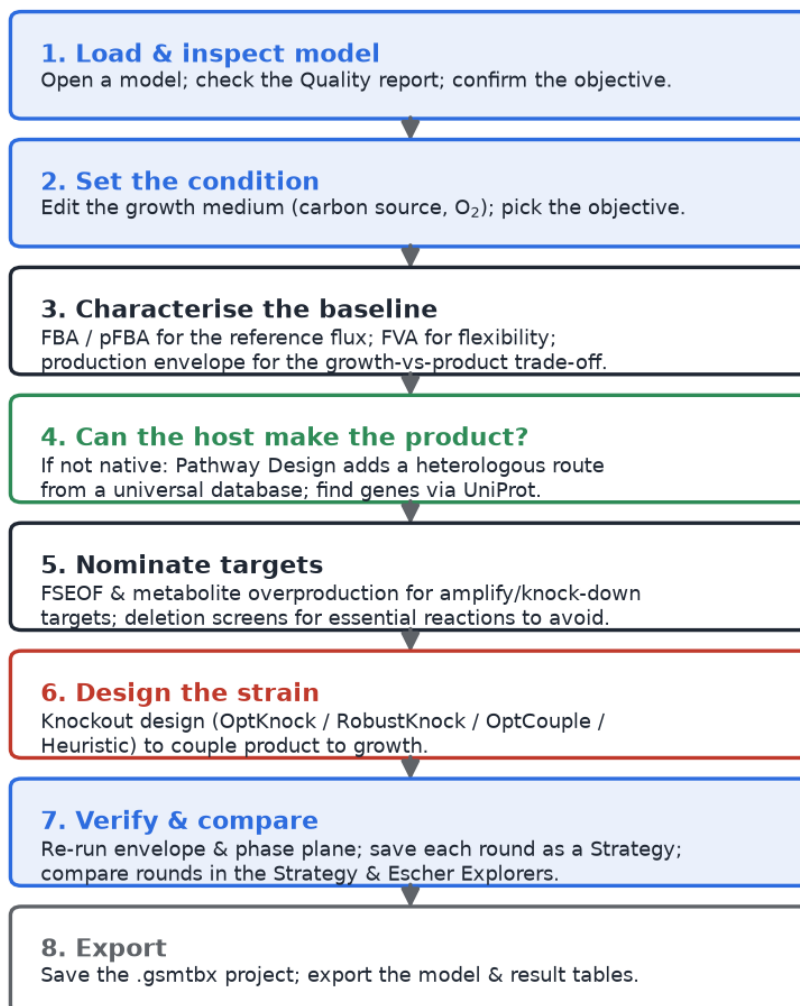


Figure 13.1. The end-to-end workflow, from loading a model to an exported, improved design.

13.1 Characterise the baseline

- 1 Load and check.** Open the model (Chapter 3), run the **Quality report** (Chapter 8) and fix any glaring imbalances. Confirm the **objective** is the biomass reaction.
- 2 Set the condition.** Edit the **growth medium** (Chapter 4): choose the carbon source, its uptake rate, and aerobic/anaerobic. This defines the yield basis.
- 3 Reference flux.** Run **pFBA** for a clean baseline flux state and note the growth rate. Save it as a **strategy** named "Wild type".
- 4 Flexibility and trade-off.** Run **FVA** to see which reactions are flexible, and a **production envelope** for your target product to see the current growth-vs-production trade-off (Chapter 7).

13.2 Make sure the product is reachable

- 1 If the host **cannot** make the target at all, open **Pathway Design** (Chapter 10), predict a heterologous route, review the alternatives, and **add** the chosen pathway to the model. Use **Find enzymes in UniProt** to collect candidate genes.
- 2 Re-run FBA and the production envelope on the engineered host to confirm the product can now be secreted, and save a strategy named “+ pathway”.

13.3 Find and commit interventions

- 1 **Shortlist targets.** Run **FSEOF** (or **Overproduce a metabolite**) to nominate amplification and knock-down targets (Chapter 9).
- 2 **Avoid essentials.** Run the **deletion screens** (Chapter 8) so you never propose knocking out an essential reaction.
- 3 **Design knockouts.** Run **Knockout strain design** (OptKnock/RobustKnock/OptCouple, or Heuristic) with a sensible minimum-growth fraction. Prefer designs with a guaranteed product floor.
- 4 **Apply and save.** Apply the chosen knockouts, re-solve, and save a strategy (“+ knockouts”).

13.4 Verify and compare rounds

- 1 Re-draw the **production envelope**: the lower edge should now be lifted (growth-coupled). Check a **phenotypic phase plane** if oxygen or a second nutrient matters.
- 2 Open the **Strategy Visualizer** and the **Escher Visualizer** (Chapter 5). Use the **difference map** to see exactly how flux re-routed from wild type to the final design, and the **titre waterfall** to see the product gain per round.
- 3 Use the Escher **FBA navigator** to probe remaining bottlenecks. Knock a candidate reaction up or down and watch the objective and flux respond.
- 4 **Export.** Save the .gsmtbx project and export the final model and the result tables (Chapter 14).

Rule of thumb. Save a strategy after every meaningful change. The difference maps and waterfalls are only as informative as the checkpoints you leave behind, and they are what turn a pile of edits into a story you can explain to collaborators.

PART XI

Changing conditions, regulation and route chemistry

A model at one fixed condition answers one question. These chapters cover what happens as conditions change, how to constrain a model with what the organism actually does, and the chemistry checks that decide whether a designed route is real.

14. Dynamic analysis: how the network responds as conditions change

Every method so far answers a question about one fixed condition. A culture is not one condition. It consumes its medium, the light changes, a nutrient runs out. The **Dynamic Analysis** tab covers the two questions that follow from that: what happens as one variable is swept, and what happens to a batch culture over time.

14.1 Condition scan

Sweeps a single condition. The concentration of a medium component, or an organism-level quantity such as light, and solves the model at each point. The output is a table of growth and any fluxes you asked to follow, plus the **breakpoints**: the values at which the response changes sharply, which is where the network rewires. Use **Add Component** to choose what is swept and the reaction browser to choose which fluxes are reported.

14.2 Time course

Follows a batch culture as it consumes its medium. Uptake follows a Monod form of the remaining concentration, and biomass and every pool are integrated forward. Any medium component can be tracked, not a fixed list of nutrients; components are seeded from the model's main carbon and nitrogen sources and you add more as needed. A component supplied in excess can be marked **buffered**, which holds its concentration rather than draining it.

Caution. Integration is explicit Euler. It is reliable for *when* a culture changes state and not for precise kinetics; the step size is reported with the results so the reader can judge. Halve the step and re-run if a transition time matters.

14.3 Stored runs, tables and plots

Each completed run is stored under its own tab at the bottom of the panel, named ConditionScan_1 or TimeCourse_1. Runs accumulate rather than replacing one another, because the comparison between two runs is usually the reason for running the second. Click a tab to open that run's table in its own window; double-click to rename it; the ✕ discards it.

- **Run** starts whichever analysis tab is open. The button names it.
- **Display Table** opens the selected run's results.
- **Plot Results** opens the axis chooser over every stored run.
- **Regulation** applies the active rule set to the run; the button beside it opens the rule-set manager.

14.4 Overlaying runs on one plot

The plot dialog takes any number of stored runs and draws them on shared axes. **Colour is the quantity and line style is the run**, so the eye reads “same measurement” from the colour and “which run” from the dashes. The X axis is free rather than fixed to time: plotting a flux against a remaining nutrient answers what the cell does as that nutrient runs out, directly.

Note. Axis options are the union across the selected runs. A run that lacks a column is **reported as missing** rather than drawn as a line at zero. absent and zero are opposite claims, and conflating them has produced confidently wrong readings before.

15. Regulation: constraining a model with what the cell actually does

A stoichiometric model will do anything the stoichiometry allows. It will run the Calvin cycle in the dark, keep photosynthesising after nitrogen has run out, and treat a carbon-limited cell as though its carbon-concentrating mechanism were already induced. A **rule set** adds the layer that stops it: sensors read the condition, a response function converts that into an activation, and an effect changes the model.

15.1 Rule sets are files you load

Nothing is applied unless you make a set active. Regulation describes one organism, and applying the wrong organism's rules produces confident, wrong answers. Load a set from *Tools > Regulation*; imported sets are stored and offered again on later launches.

15.2 What a rule can do

Effect	What it changes	When it binds
gate	Closes a reaction below a threshold	Always. A closed gate is a hard constraint
scale	Multiplies a flux bound	Only if that bound was limiting
enzyme_cost	Changes a reaction's protein cost	Enzyme-constrained models only
budget	Resizes the whole protein pool	Enzyme-constrained models only
biomass	Reweights a biomass component	Always
parameter	Records a named quantity	Not applied to the network

15.3 Check a rule set before you trust it

Selecting a rule set reports how much of it can act on the model you have open. for example “5 of 12 rules can change this model; 2 use an unapplied effect; 5 only raise a capacity”. This matters more than it sounds. A rule can be reported as firing, with a magnitude, while changing no bound and no flux: a target that does not exist under that name, an effect the simulation does not apply, or a ceiling raised above a flux that never reached it.

Why it matters. A rule set written for one model rarely transfers unchanged to another. Reaction ids differ, and the two protein-allocation effects need an enzyme-constrained model. On a plain reconstruction they run without error and change nothing. Read the audit before reading the results.

15.4 The rule editor

Build or edit a set against the model you have open. Each rule records what it senses, how it responds (step, Hill or ramp), what it affects, and. Importantly. its **provenance and confidence**: measured, inferred or assumed. A conclusion resting on assumed thresholds is flagged in the results, which is the honest way to present a threshold nobody has measured.

16. Pathway design: rule-based chemistry, enzymes and feasibility

16.1 When the databases have no route

A database search can only return chemistry somebody has already catalogued. **Rule-based (RetroRules) search** instead applies reaction rules to your target's structure, proposing steps that no database lists. These are **predictions to verify**, not database entries. The search samples a very large rule set in an order fixed by a seed, so re-running returns the same routes and a different seed may surface different, equally valid ones. The seed is reported with the results and should be quoted when sharing them.

16.2 Finding an enzyme

A proposed step is useless without a gene to express. Where a reaction carries an EC number, candidate enzymes are looked up directly. Where it does not. The normal case for rule-generated chemistry. **reaction-similarity search** finds characterised reactions with similar chemistry and returns their enzymes, ranked by similarity, curation status and taxonomic proximity to your host. A loose match is flagged as such.

16.3 Thermodynamic feasibility

The **max-min driving force** analysis asks whether a route can run at all: it finds the metabolite concentrations that leave the least-favourable step as favourable as possible. A route whose best case is still negative will not carry flux however well expressed. Enable the suite in Preferences; the thermodynamic data downloads once, on consent.

16.4 Chemistry checks on a proposed route

Mass and charge balance are checked on every step, but balance cannot see a rearranged carbon skeleton. A reduction that turns a branched aldehyde into a straight-chain alcohol balances perfectly and is still impossible. Routes are therefore also checked for **skeletal-isomer swaps**: a substrate and product with the same carbon count but different backbones, where atoms are added or removed as well.

Caution. Where the compounds carry structures the verdict is definite and reads *Impossible*. Where they do not, the skeleton is inferred from nomenclature and reads *Possible*. A suspicion to confirm, never a reason to discard a route unexamined. Database annotation errors of exactly this kind do occur in public reaction databases.

17. Tasks, performance, results and settings

17.1 Background tasks and concurrency

Every long task (an analysis, a database download, a UniProt lookup) runs in the background, and the **status bar** shows a specific description plus an estimated-progress bar whose estimate is learned from previous runs. You can launch **several analyses at once**. Up to four run concurrently and the interface stays usable; when more than one is running the status bar reads “Multiple analyses running (N)”, and the arrow at the bottom-right opens a window listing each task with its own progress and an **×** button to **cancel** it. Launch more than four and the extras are **queued** automatically.

17.2 A note on speed and CPU usage

The constraint-based solvers (GLPK for linear programs, SCIP for the mixed-integer strain-design problems) are exact and single-threaded, so a single analysis uses about one CPU core. Modest total CPU on a multi-core machine is expected and is *not* a bottleneck. Splitting one analysis across many cores is actually counter-productive on Windows, where starting worker processes costs more than the solves; the toolbox deliberately uses the fast single-process path. To use several cores, run several analyses at once. Each in its own process. There is no GPU acceleration because LP/MILP solvers do not benefit from a GPU; the limits are single-core CPU speed and, for very large models, memory.

17.3 Results folder and saving

Settings > Set results output folder (or *View > Set results output folder*) chooses a folder where every result table is also saved automatically as a timestamped CSV. The **Save Results** button beneath each analysis writes the current view in the fitting format. A table as CSV, a plot as PNG and PDF. If no folder is set, you are prompted to pick one.

17.4 Settings and the offline cache

Settings > Manage Data shows everything the app has downloaded. reaction databases, the MetaNetX reference files, and molecule 2-D structure images, all stored permanently under `~/.gsm_toolbox` and reused on later runs, so the app keeps working **offline** and never re-downloads the same data. The window lists cached files by category with their sizes and lets you delete selected files or clear a whole category to free space; anything removed is simply fetched again if needed later.

Settings

Refresh

Stored cache. Downloaded reaction databases and molecule structure images are kept on disk so the app works offline and doesn't re-download them. Remove files you no longer need to free space — they will be fetched again if required.

Cache location: C:\Users\fmjor\.gsm_toolbox

Reaction databases

metanetx_chem_xref.tsv (647.1 MB)
metanetx_reac_xref.tsv (77.0 MB)
bigg_universal_model.json (20.5 MB)
metanetx_universal_enzymatic_named2.json (10.9 MB)
metanetx_reac_prop.tsv (9.8 MB)

10 file(s) · 779.8 MB

Delete selected

Clear all in this category

Molecule structures

19c01301b3581923f7447a45c762e7c68b14a5ae.png (4.0 KB)
513fa7dc84ec3c931ed3b56db42b82ca523ae77d.png (3.6 KB)

Figure 17.1. Settings > Manage Data: the persistent cache, grouped by category, with per-file sizes and controls to free space.

Appendix A. Method comparison at a glance

The table summarises what each method needs, what it returns, and the engineering question it answers. “Cost” is the rough compute effort.

Method	Key input	Output	Question answered	Cost
FBA	Objective	Flux vector, objective value	What is the optimal flux state?	Low
pFBA	Objective	Parsimonious flux vector	Most efficient optimal fluxes?	Low
FVA	Objective fraction	Min/max flux per reaction	Which fluxes are flexible?	Medium
Shadow prices	FBA solution	Dual values per metabolite	What limits the objective?	Low
MOMA / ROOM	Knockouts	Mutant flux state	How does a fresh mutant behave?	Medium
Deletion screens	,	Growth per knockout	Which genes/reactions are essential?	Medium
Production envelope	Product reaction	Growth-vs-product curve	Growth/production trade-off?	Medium
Robustness	Control reaction	Objective vs fixed flux	Where are the tipping points?	Medium
Phase plane	Two reactions	Objective heatmap	How do two inputs interact?	Medium
Flux sampling	,	Per-reaction distributions	How is flux distributed?	High
Quality report	,	Scorecard + issue lists	Is the model well-formed?	Medium
Blocked reactions	,	List of dead reactions	What cannot carry flux?	Medium
Gap-filling	Universal model, target	Reactions to add	What is missing to enable X?	Medium
OptKnock	Product, #KOs	Knockout strategies	Which KOs couple product to growth?	High
RobustKnock	Product, #KOs	Knockout strategies	Which KOs guarantee production?	High
OptCouple	Product, #KOs	Knockout strategies	Which KOs maximise coupling?	High
FSEOF	Product reaction	Over/under-expression targets	What to up/down-regulate?	Low
Overproduce metabolite	Target metabolite	Amplify/knock-down + direction	How to accumulate a compound?	Low
eFlux	Expression table	Context-specific fluxes	What does the cell do given expression?	Low
GIMME	Expression + threshold	Context fluxes + score	Flux consistent with expression?	Medium
ATPM sensitivity	ATPM reaction	Growth vs maintenance	How much energy burden is tolerable?	Low
Elementary flux modes	Small category	List of minimal pathways	What are the fundamental routes?	High
Community modelling	2+ models	Per-member growth	How do organisms grow together?	Medium

Method	Key input	Output	Question answered	Cost
Pathway design	Target + database	Reactions to add	How to make a non-native product?	Medium

Appendix B. Glossary

Biomass reaction. A pseudo-reaction that drains precursors in the proportions needed to build new cell mass; optimising its flux predicts the growth rate.

Constraint-based modelling. Predicting the space of feasible flux states from mass balance and capacity constraints, without kinetic parameters.

Currency metabolite. A ubiquitous cofactor/carrier (ATP, NAD(P)H, water, protons, CoA, ...) that appears in many reactions; hidden by default in maps for legibility.

Exchange reaction. A boundary reaction that lets a metabolite enter or leave the system; the growth medium is defined by the uptake bounds on exchanges.

FBA. Flux Balance Analysis: linear optimisation of an objective over the feasible flux space.

Flux. The rate at which a reaction proceeds, in mmol per gram dry weight per hour ($\text{mmol} \cdot \text{gDW}^{-1} \cdot \text{h}^{-1}$); the model's basic unknown.

Gene–reaction rule (GPR). A Boolean expression linking a reaction to the genes required to catalyse it (AND = complex, OR = isozymes).

Growth-coupling. A design in which producing the target is unavoidable when the cell grows fast, so selection for growth also selects for production.

Objective. The reaction (or weighted combination) that a simulation maximises; usually biomass, sometimes a product.

pFBA. Parsimonious FBA: among optimal solutions, the one minimising total flux.

Shadow price. The change in the objective per unit change in a metabolite's availability; flags limiting metabolites.

Steady state. The assumption that internal metabolite concentrations do not change, so production equals consumption for every internal metabolite ($S \cdot v = 0$).

Strategy. A named, solved flux state saved after an engineering round, for comparison in the Strategy and Escher Visualizers.

Stoichiometric matrix (S). The table of reaction stoichiometries; its columns are reactions and its rows are metabolites.

Universal reaction database. A large collection of known biochemistry (BiGG, MetaNetX, ModelSEED, KEGG) used to gap-fill or design heterologous pathways.

Appendix C. Troubleshooting and FAQ

FBA returns zero growth. What is wrong?

Almost always the medium: a required nutrient (carbon, nitrogen, phosphate, a needed ion) has its uptake closed, or oxygen is off when the model needs it. Open the growth-medium editor and check the exchange bounds. If the medium is right, run the Quality report and Blocked reactions. A gap in biomass precursor synthesis can also abolish growth, and gap-filling can repair it.

The predicted growth rate looks too high / too low.

Check the carbon-source uptake rate (the yield basis) and whether oxygen is available. Unrealistically high growth often means an unconstrained or duplicated exchange; too low often means an over-tight bound or a missing reaction. FVA and the Quality report help locate the cause.

Strain design finds no solutions.

Loosen the constraints: lower the minimum-growth fraction (e.g. to 0.01), allow more knockouts, choose the “Any/Diverse” approach, or switch to the Heuristic method. Confirm first (with a production envelope) that the product can be made at all in the current model. If not, design a pathway first (Chapter 10).

Pathway design says the target is unreachable.

Load a broader database (the MetaNetX universal covers the most chemistry), accept the MetaNetX reference download for the best cross-identifier matching, and relax the search (more starting metabolites, a larger maximum length). Also confirm the target metabolite you picked is the one you meant. The list shows names alongside ids.

A downloaded database is taking a long time the first time.

Genome-scale universals are large (tens of thousands of reactions) and are parsed and indexed once; the result is cached permanently, so later launches load it quickly. You can watch progress in the status bar and cancel from the tasks window.

Why do currency metabolites not appear in the map?

ATP, water, protons and other ubiquitous carriers connect to hundreds of reactions and would turn the map into a hairball, so they are hidden by default. They are still in the model and in every calculation. Only the drawing omits them.

Can I use a commercial solver?

Yes. GLPK and SCIP are bundled and used automatically. If a supported commercial solver is installed in the environment it can be selected in the strain-design dialog's solver dropdown for larger mixed-integer problems.

Appendix D. File formats and where data is stored

- **Models** load and export as SBML (.xml/.sbml), JSON, or MATLAB (.mat).
- **Projects** save as .gsmtbx. A single file holding the edited model, categories, saved strategies and settings.
- **Designed pathways** can be saved/loaded as .gsmath JSON files.
- **Result tables** export as CSV; **plots** as PNG and PDF; **maps** as PNG (Network Map) or SVG/PNG (Escher).
- **Downloads and caches** live under ~/.gsm_toolbox (reaction databases, reference files, structure images, learned task durations).

Appendix E. Worked examples

These two end-to-end case studies show how the pieces of the toolbox combine on realistic problems. They are written as recipes you can follow with your own models; the exact numbers depend on the model versions and media you use, and some steps depend on external databases that evolve over time. Where a step exposed a rough edge in the current toolbox, it is flagged “**To improve**” so the limitation is transparent.

E.1 A light-driven CO₂-to-styrene consortium

This tutorial builds, from scratch, a two-organism community that turns **light and CO₂ into styrene**. A bulk polymer precursor that is toxic to most cells. No single organism does this well, so we split the job: a cyanobacterium fixes CO₂ and secretes acetate (and O₂) in the light, and a robust soil bacterium eats that acetate to make styrene, returning CO₂ that the cyanobacterium re-fixes. You will engineer each strain, join them into a consortium, and quantify the cross-feeding. Every number below was produced by the toolbox on the two published models; yours will match if you use the same models and media.

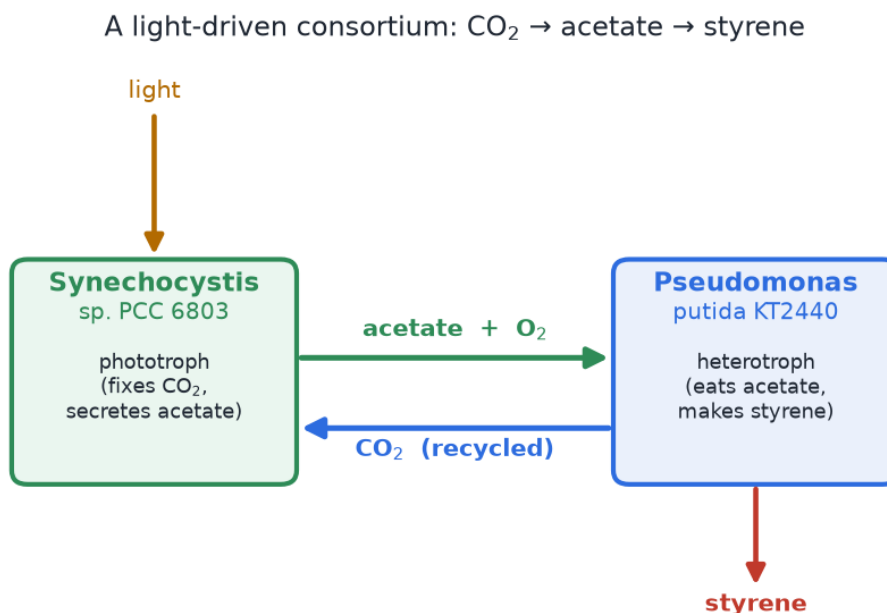


Figure E.1. The design: light drives CO₂ fixation and acetate/O₂ export by PCC 6803; P. Putida consumes acetate and O₂ to make styrene and returns CO₂. The carbon and oxygen are largely recycled inside the co-culture.

Models. No models ship with the application; both are fetched. The phototroph is iJN678, the genome-scale reconstruction of *Synechocystis* sp. PCC 6803 [25] (863 reactions). The heterotroph is iJN1463, the high-quality model of *Pseudomonas putida* KT2440 [27] (2,927 reactions); fetch it as a BiGG model by id from the Pathway Design tab, or download it once from bigg.ucsd.edu. Both use BiGG identifiers, so their shared exchange metabolites (acetate ac_e, CO₂, O₂) line up automatically when the community is built.

Phase 1. Characterise the phototroph

- 1 **Fetch** iJN678 by id from the Pathway Design tab, or download it from bigg.ucsd.edu and open it with *File > Open Model*. It carries three biomass reactions: BIOMASS_Ec_SynAuto (photoautotrophic), ...SynMixo (mixotrophic) and ...SynHetero (heterotrophic). The default objective, which grows the model on glucose in the dark).

- 2 **Switch to photoautotrophy.** In Growth Settings choose the autotrophic growth mode (or by hand: set the objective to BIOMASS_Ec_SynAuto, open photon uptake EX_photon_e to -100, allow CO₂ uptake, and close the glucose exchange EX_glc__D_e).
- 3 **Run FBA.** The wild type grows at **0.163 h⁻¹**, taking up 6.84 units of CO₂ and **evolving 10.3 units of O₂**. The signature of oxygenic photosynthesis. Save this as a strategy “PCC6803 WT”.

Why it matters. This baseline is the reference every later state is compared against. Note the two things the phototroph gives the consortium for free: fixed carbon and oxygen.

Phase 2. Engineer acetate overproduction

For the phototroph to feed its partner it must **secrete acetate** while still growing. Run a **production envelope** with the target set to the acetate exchange EX_ac_e: the toolbox scans acetate secretion and reports the growth still achievable at each level (Figure E.2, left). The trade-off is linear. At the maximum acetate flux (5.0) growth is zero; at zero acetate the cell grows fastest. To nominate the interventions that push carbon to acetate, run **Overproduce a metabolite** (target acetate) or **FSEOF** on EX_ac_e.

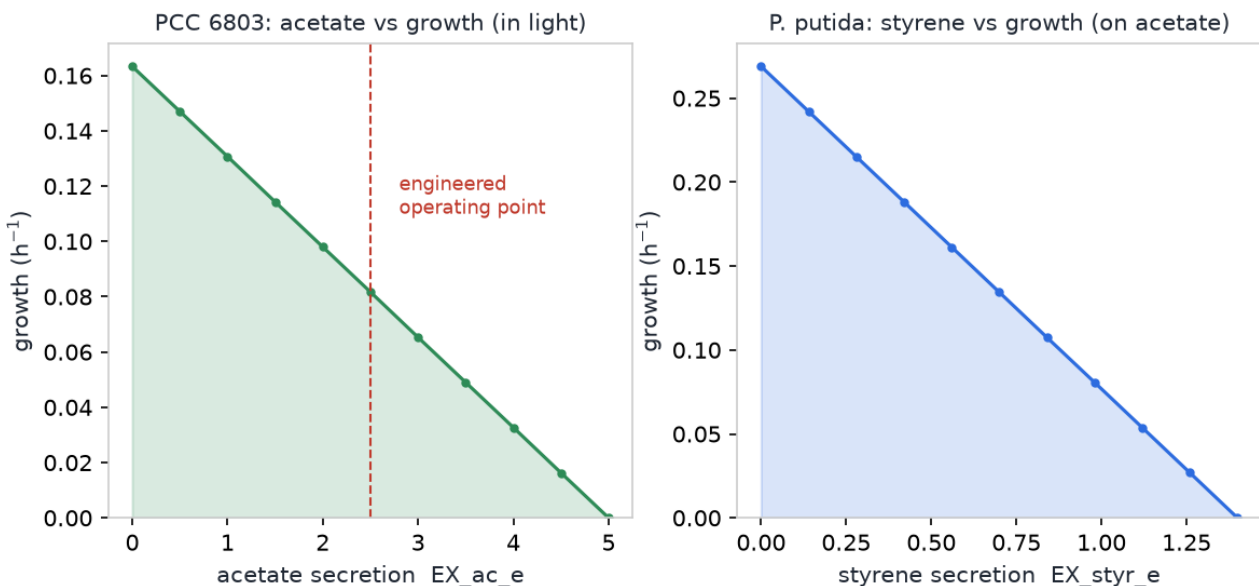


Figure E.2. Production envelopes. Left: PCC 6803 growth vs acetate secretion in the light. The red line marks the operating point used here (acetate = 2.5, growth = 0.082). Right: P. Putida growth vs styrene secretion on acetate.

- 1 Pick an **operating point** on the envelope. Here we require acetate secretion of 2.5 (half the ceiling). The engineered strain then grows at **0.082 h⁻¹** while secreting acetate, and fixes even more CO₂ (8.4) to pay for it.
- 2 Save this state as a strategy “PCC6803 acetate+”. In the **Strategy Visualizer** you can now draw the difference map “acetate+ vs WT” to see carbon re-routing from biomass toward acetate export.

Phase 3. Build the styrene pathway in P. Putida

Styrene is not native to *P. Putida*. The established two-step route is L-phenylalanine → *trans*-cinnamate (phenylalanine ammonia-lyase, PAL, EC 4.3.1.24) → styrene (*trans*-cinnamate decarboxylase, e.g. ferulic-acid decarboxylase FDC1, EC 4.1.1.-). *P. Putida* already makes phenylalanine, so only these two heterologous steps are needed.

- 1 Open the loaded *P. Putida* model and set the medium to **acetate as the sole carbon source** (uptake 10) plus O₂. Confirm with FBA that it grows: **0.269 h⁻¹**, consuming 8.73 O₂ and **releasing 9.17 CO₂**. The mirror image of the phototroph.
- 2 In **Pathway Design**, target **styrene** and predict the route against the MetaNetX universal (broadest coverage of aromatics); it recovers the PAL + decarboxylase steps. **Add the pathway**. The summary dialog reports two new reactions, both mass-balanced, and , because the toolbox always guarantees a way out for the product. A new styrene exchange EX_styr_e is created automatically.
- 3 Use **Find enzymes in UniProt** on the EC numbers to pick genes to express (a plant/microbial PAL such as AtPAL2, and a fungal FDC1).
- 4 Draw the new route in the **Escher Visualizer**: focus on the styrene category and you get the clean linear pathway of Figure E.3, with the flux overlaid on each step.

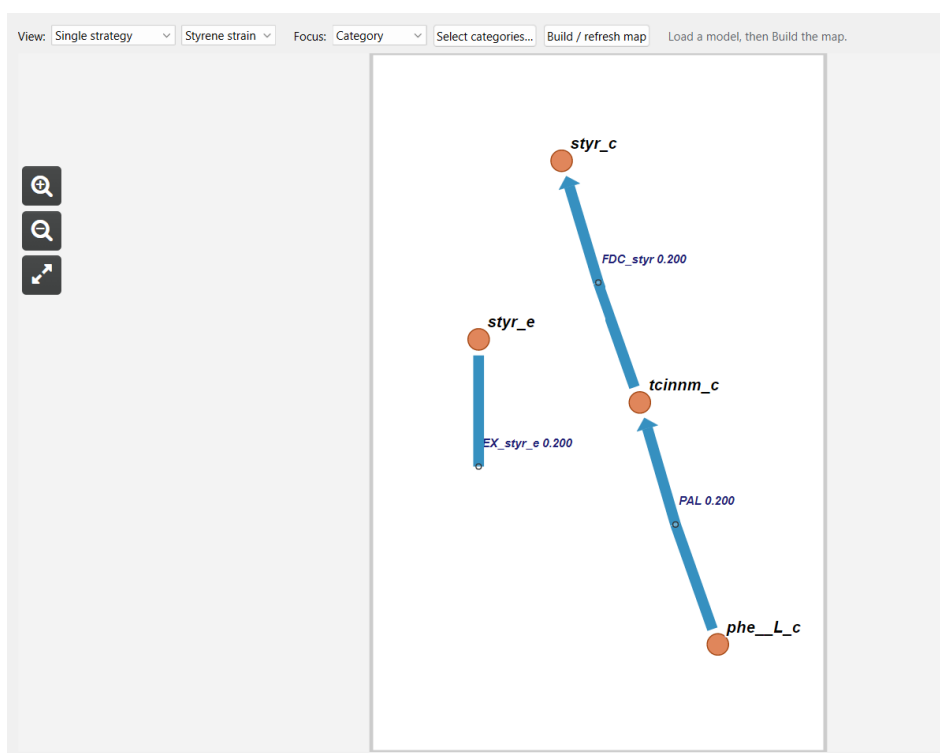


Figure E.3. The heterologous styrene pathway in *P. Putida*, drawn in the Escher Visualizer (Focus > Category): phenylalanine -> PAL -> trans-cinnamate -> FDC -> styrene -> exchange, with flux (0.20) shown on each arrow.

A styrene **production envelope** (Figure E.2, right) shows the growth/styrene trade-off on acetate: the maximum styrene flux is **1.40** at zero growth, and any intermediate point trades biomass for product. Save the engineered strain as “Pputida styrene+”.

Phase 4. Join them into a consortium

- 1 Open **Tools > Consortia modelling > Build Community Model** and add the two engineered strains. The builder tags each organism's internal reactions, joins them through a shared external pool, and produces a single community model (here 3,750 reactions).
- 2 Set the community medium: **light and CO₂ available**, O₂ free to move, and **no external organic carbon**, so the only acetate in the system is what PCC 6803 secretes, forcing the cross-feed.
- 3 Open **Consortia objective** (in the objective settings) and set a **minimum growth per member** of 0.3. This is the key step: without it, community FBA maximises the summed biomass and lets the faster grower take everything; with it, each organism must grow, which *forces* 6803 to feed acetate to putida.

- 4 Run **Community member growth**. Both organisms grow. PCC 6803 at 0.154 and P. Putida at 0.055 h^{-1} (Figure E.4). Neither is starved.

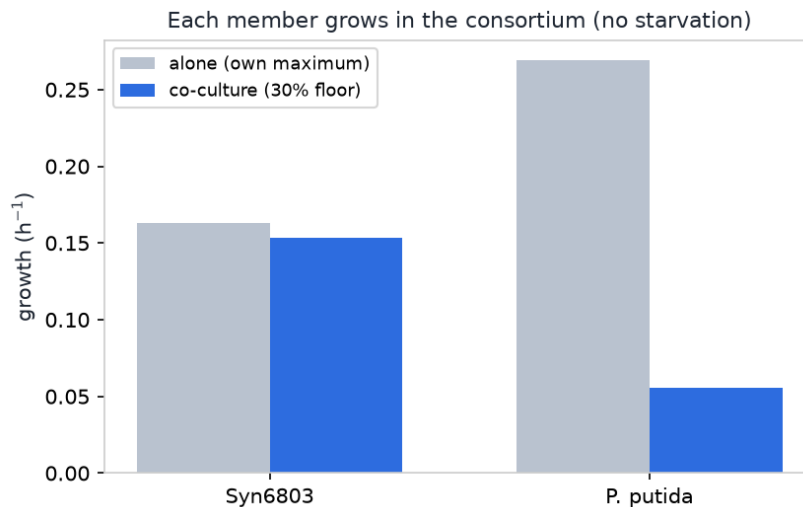


Figure E.4. Each member grows in the consortium. With the no-starvation floor the faster grower cannot monopolise the shared nutrients.

Phase 5. Analyse the cross-feeding

Now compare what crosses the system boundary when the organisms are grown **separately** versus **together** (Figure E.5). This is the payoff of the whole design.

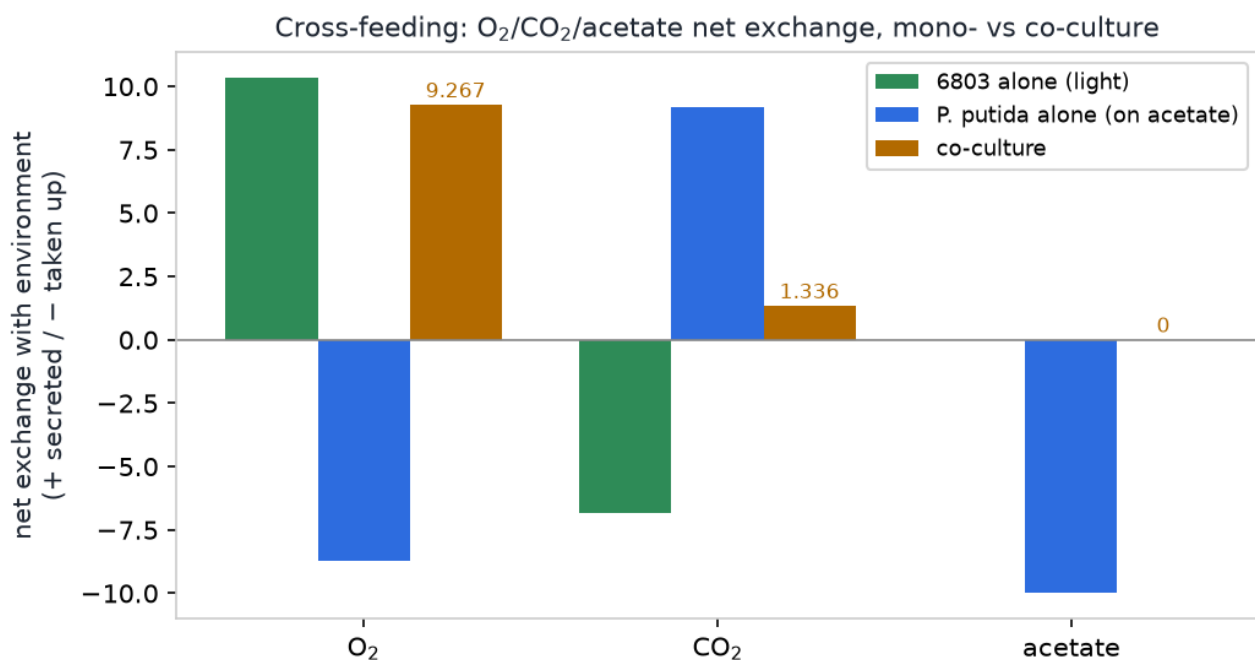


Figure E.5. Net exchange with the environment (positive = secreted, negative = taken up) for O_2 , CO_2 and acetate, in mono-culture and co-culture.

- **Acetate** goes from -10 (P. Putida alone must be fed it externally) to 0 in the co-culture. It is produced by 6803 and consumed by putida entirely inside the community, never touching the environment.
- **CO_2** drops from +9.17 (putida alone respire it away) to just +1.34 in the co-culture: most of the heterotroph's CO_2 is re-fixed by the phototroph. A near-closed carbon loop.

- O_2 that 6803 evolves is partly consumed by putida, so the net (+9.27) is the phototroph's output minus the heterotroph's demand. The oxygen hand-off that lets putida respire without external aeration.

And the point of it all: maximising the styrene exchange in the community shows the consortium can make styrene at up to **0.42 flux from light and CO_2 alone**, with the entire carbon chain (CO_2 -> acetate -> phenylalanine -> styrene) running internally. Use the **Escher Visualizer** difference map (co-culture vs the phototroph alone) to see how flux re-routes, and the **Strategy Visualizer** titre waterfall to read the styrene gain across the engineering rounds.

E.1. Comparing the states at a glance

Every round was saved as a strategy; the table summarises them.

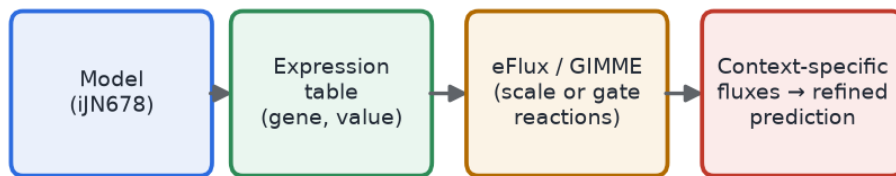
State	Growth (h^{-1})	Key exchange fluxes	Role
PCC6803 WT (light)	0.163	O_2 +10.3, CO_2 -6.8, acetate 0	reference phototroph
PCC6803 acetate+	0.082	acetate +2.5, CO_2 -8.4	acetate donor
P. Putida on acetate	0.269	acetate -10, O_2 -8.7, CO_2 +9.2	acetate consumer
P. Putida styrene+	≤ 0.269	styrene up to 1.40	styrene producer
Consortium (co-culture)	0.154 / 0.055	acetate 0, CO_2 +1.3, styrene ≤ 0.42	closed CO_2 -to-styrene loop

Note. The consortium's low individual growth rates are expected: photoautotrophic growth is inherently slow, and the community solution shares a fixed light/ CO_2 budget between two organisms plus product. The value of the co-culture is the conversion it enables (light + CO_2 -> styrene, carbon recycled), not the growth rate.

E.2 Integrating omics data to refine predictions

Plain FBA assumes any reaction can run up to its stoichiometric limit, so it predicts the same optimistic flux state regardless of what the cell is actually doing. This tutorial folds measured **gene expression** into the model to make the prediction **condition-specific**, using the classic response of *Synechocystis* sp. PCC 6803 to **nitrogen starvation**: the cell throttles photosynthesis and growth and shifts carbon toward storage. We integrate **transcriptomics (RNA-seq)** and **proteomics** separately, ask whether they agree, and then combine them.

Omics integration: from an expression table to a refined flux prediction



gene ids in the table must match the model's locus tags (e.g. *slr/slr...*)

Figure E.6. Omics integration: an expression table is mapped onto the model's genes, which scales the reactions and yields a condition-specific flux state.

Finding the data

Synechocystis sp. PCC 6803 is among the most data-rich phototrophs, and nitrogen starvation is one of its best-characterised conditions. Freely available, matched resources include:

- **Models.** iJN678 [25] (used here; 622 genes) and the larger iSynCJ816 [26]; both key genes by the strain's *slr/sll* locus tags.
- **Transcriptomics.** Nitrogen-depletion RNA-seq time courses deposited in NCBI GEO; the primary-transcriptome map of PCC 6803 is also published.
- **Proteomics.** Quantitative (label-free / iTRAQ) proteomes across nutrient conditions and growth rates in PRIDE, useful because protein abundance constrains enzyme capacity directly.

Note. The RNA-seq and proteome tables used in this walk-through are a **representative dataset** constructed to reproduce the documented N-starvation response (photosynthesis and growth genes down; storage and central-carbon genes up), keyed to the model's actual *slr/sll* genes, so the workflow and the toolbox behaviour are exactly what you get with measured data. Substitute a real GEO/PRIDE table and every step is identical.

Preparing and mapping each dataset

Raw omics tables almost never arrive as (model gene id, value). Use *Analysis > Omics & energy > Prepare omics dataset* for each file. Figure E.7 shows the wizard on the RNA-seq table: pick the data type (*Transcriptomics*), the identifier column (*Locus*) and the value column, then **Map to model**.

Load a raw omics table in any common format (CSV / TSV / Excel). The tool maps its identifiers onto this model and produces the (id, value) data used for context-specific analysis.

1 · Source file and data type

Choose file... *no file chosen*

Data type: Transcriptomics (RNA-seq / microarray → genes) ▼

2 · Columns

Identifier column: locus ▼ Combine samples by: mean ▼

Value column(s) — tick the samples/replicates to use:

☐ locus

☒ value (numeric)

Map to model

3 · Mapping summary

Data type: transcriptomics
 Identifier column: locus
 Value column(s): value (aggregate: mean)
 Source rows: 622 | mapped to the model: 622
 Model genes covered: 622 / 622 (100.0%)
 Unmatched identifiers: 0

Use this dataset Save prepared table... Cancel

Figure E.7. The Dataset Preparation wizard on the N-starvation RNA-seq table. Because the identifiers are the model's own locus tags, all 622 genes map (100% coverage). Proteomics is prepared the same way.

Repeat for the proteome table (data type *Proteomics*). As soon as the first dataset is mapped, an **Omics** tab appears in the main window (Figure E.8) listing every prepared dataset, its coverage summary, and the per-gene values that will feed eFlux and GIMME.

Omics datasets mapped to the model			Dataset: proteome (N-starved)
Data type: proteomics Identifier column: locus Value column(s): value (aggregate: mean) Source rows: 622 mapped to the model: 622 Model genes covered: 622 / 622 (100.0%) Unmatched identifiers: 0			
	gene	value	
1	zwf	62.5905	
2	zntC	176.288	
3	yibO	159.006	
4	yefA	71.1757	
5	ycf6	32.4014	
6	ycf40	137.214	
7	ycf32	97.6802	
8	valS	43.5563	
9	ureG	42.5519	
10	ureF	47.1257	
11	ureE	114.444	

Figure E.8. The Omics tab. Visible only once omics data has been mapped. Showing the prepared proteome dataset (100% coverage) and its values.

Do the two omics agree?

Before trusting either dataset, compare them. Plotting the mapped RNA-seq value against the proteome value for every gene (Figure E.9) gives a **Pearson correlation of 0.86** in log space. The transcriptome and proteome agree well on *which* genes are high or low under nitrogen starvation. This is reassuring, but note the scatter: individual genes can differ severalfold between mRNA and protein, which matters below.

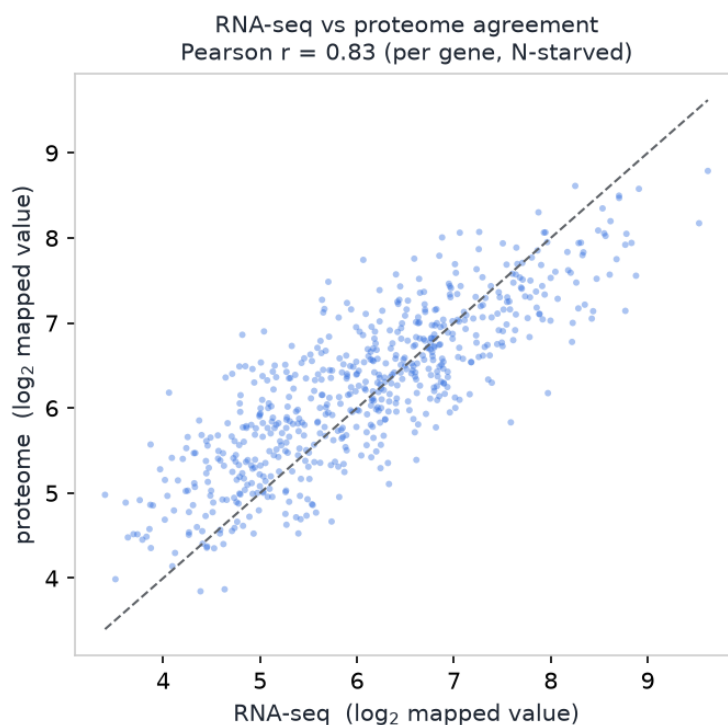


Figure E.9. Gene-level agreement between the RNA-seq and proteome datasets (Pearson $r = 0.86$, $\log_2$ values). The dashed line is $y = x$.

Refining the prediction with each omics type

Now run **eFlux** with each dataset (it scales every reaction's capacity by the expression of its genes) and compare the predicted flux state to plain FBA. Figure E.10 shows the predicted growth rate and photosynthetic activity (O_2 evolution) for four cases.

Omics makes the prediction condition-specific (transcriptome shifts most, proteome is buffered)

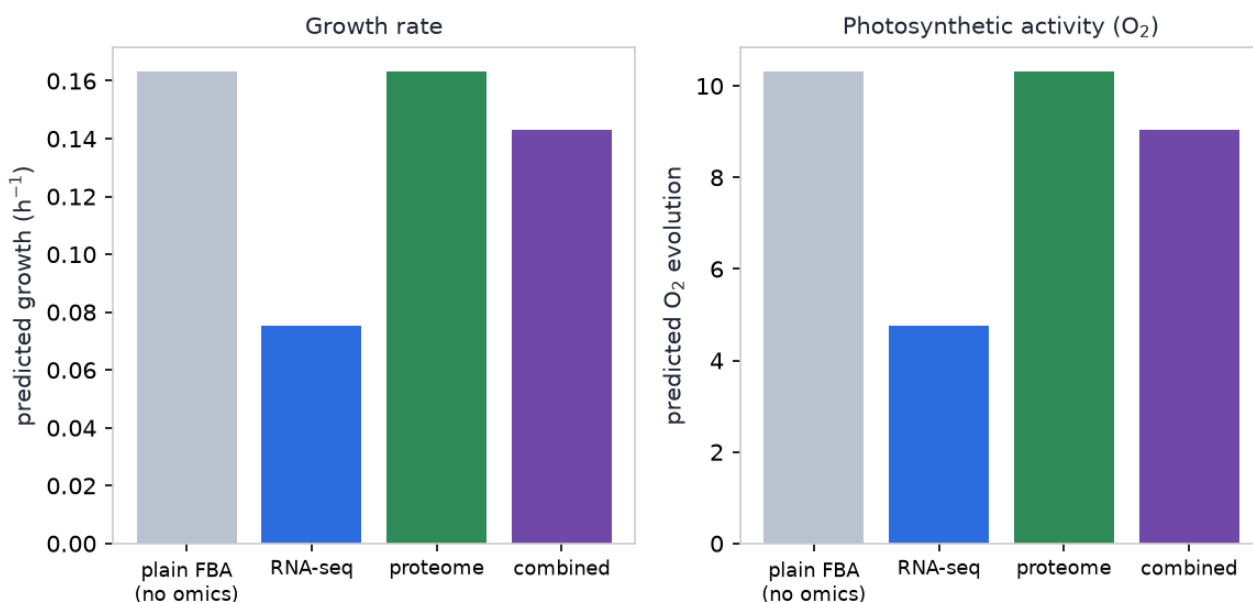


Figure E.10. Predicted growth and O_2 evolution: plain FBA (no omics) vs eFlux constrained by RNA-seq, by the proteome, and by the two combined. Omics makes the prediction condition-specific.

- **Plain FBA** ignores the condition and predicts the cell runs at full tilt (growth 0.163, O_2 10.3). The same as a nutrient-replete cell.
- **RNA-seq** imposes the strong transcriptional down-regulation of photosynthesis and growth genes, so eFlux predicts a large drop: growth **-54%** (0.075) and O_2 evolution roughly halved (4.8).
- **Proteome** gives the same *direction* but a much smaller drop (growth -6%, 0.153): protein levels are **buffered**. They change less than the transcripts, a well-known feature of cyanobacterial physiology. The two omics agree on what happens but not on how strongly.
- **Combined** (the consensus of both) lands in between (growth -20%, 0.131). A robust middle estimate that neither over-trusts the volatile transcriptome nor the sluggish proteome.

The eFlux flux states from RNA-seq and proteome are themselves highly correlated ($r = 0.98$ across internal reactions), confirming that both omics types steer the model toward the same physiology. A throttled, storage-leaning metabolism. Differing mainly in magnitude. **GIMME** tells the same story from the other direction, returning a lower inconsistency score under the starvation profiles because the model no longer has to force flux through genes the cell has switched off.

Combining multiple omics, and what it buys you

To combine datasets, prepare each one, then build a consensus expression table. The geometric mean of the RNA-seq and proteome values per gene is a good default (it favours genes both datasets agree are high, and down-weights disagreements). Loading the consensus and running eFlux gives the “combined” bars in Figure E.10. Combine when you have both layers for the same condition and want a single, less noisy estimate; keep them separate when you specifically want to

see where transcript and protein regulation diverge (often the most biologically interesting genes).

Refining a product prediction. The practical payoff is that any product competing with growth is now predicted in the *real* physiological state rather than the naive maximum. Here, once omics tells the model the cells are in the slow-growing, storage-leaning N-starved state, its predictions for carbon partitioning (toward glycogen and other storage) become consistent with the biology. Something plain FBA, always maximising growth, would miss. Re-run your target analysis (production envelope, FSEOF) on the expression-constrained model to get targets that are actually expressed under your condition.

Caution. The single most common failure is a **gene-identifier mismatch** between the omics table and the model (locus tags vs gene names vs UniProt/KEGG ids). The *Prepare omics dataset* tool maps across these schemes and reports the coverage; if it is low, pick a different identifier column, or a data source whose ids match your model's genes, before running eFlux/GIMME.

References

- [1] Orth JD, Thiele I, Palsson BØ. What is flux balance analysis? *Nature Biotechnology*. 2010;28(3):245–248.
- [2] Lewis NE, Hixson KK, Conrad TM, et al. Omic data from evolved *E. Coli* are consistent with computed optimal growth from genome-scale models (pFBA). *Molecular Systems Biology*. 2010;6:390.
- [3] Mahadevan R, Schilling CH. The effects of alternate optimal solutions in constraint-based genome-scale metabolic models (FVA). *Metabolic Engineering*. 2003;5(4):264–276.
- [4] Edwards JS, Palsson BØ. The *Escherichia coli* MG1655 in silico metabolic genotype: its definition, characteristics, and capabilities. *PNAS*. 2000;97(10):5528–5533.
- [5] Edwards JS, Ibarra RU, Palsson BØ. In silico predictions of *Escherichia coli* metabolic capabilities are consistent with experimental data. *Nature Biotechnology*. 2001;19(2):125–130.
- [6] Ebrahim A, Lerman JA, Palsson BØ, Hyduke DR. COBRApy: CONstraints-Based Reconstruction and Analysis for Python. *BMC Systems Biology*. 2013;7:74.
- [7] Burgard AP, Pharkya P, Maranas CD. OptKnock: a bilevel programming framework for identifying gene knockout strategies for microbial strain optimization. *Biotechnology and Bioengineering*. 2003;84(6):647–657.
- [8] Tepper N, Shlomi T. Predicting metabolic engineering knockout strategies for chemical production: accounting for competing pathways (RobustKnock). *Bioinformatics*. 2010;26(4):536–543.
- [9] Choi HS, Lee SY, Kim TY, Woo HM. In silico identification of gene amplification targets for improvement of lycopene production (FSEOF). *Applied and Environmental Microbiology*. 2010;76(10):3097–3105.
- [10] Jensen K, Broeken V, Hansen ASL, et al. OptCouple: joint simulation of gene knockouts, insertions and medium modifications for prediction of growth-coupled strain designs. *Metabolic Engineering Communications*. 2019;8:e00087.
- [11] Klamt S, Gilles ED. Minimal cut sets in biochemical reaction networks. *Bioinformatics*. 2004;20(2):226–234.
- [12] Reed JL, Patel TR, Chen KH, et al. Systems approach to refining genome annotation (gap-filling). *PNAS*. 2006;103(46):17480–17484.
- [13] Lieven C, Beber ME, Olivier BG, et al. MEMOTE for standardized genome-scale metabolic model testing. *Nature Biotechnology*. 2020;38(3):272–276.
- [14] Colijn C, Brandes A, Zucker J, et al. Interpreting expression data with metabolic flux models (eFlux). *PLoS Computational Biology*. 2009;5(8):e1000489.
- [15] Becker SA, Palsson BØ. Context-specific metabolic networks are consistent with experiments (GIMME). *PLoS Computational Biology*. 2008;4(5):e1000082.
- [16] Schuster S, Hilgetag C. On elementary flux modes in biochemical reaction systems at steady state. *Journal of Biological Systems*. 1994;2(2):165–182.
- [17] Stolyar S, Van Dien S, Hillesland KL, et al. Metabolic modeling of a mutualistic microbial community. *Molecular Systems Biology*. 2007;3:92.
- [18] Pharkya P, Burgard AP, Maranas CD. OptStrain: a computational framework for redesign of microbial production systems. *Genome Research*. 2004;14(11):2367–2376.
- [19] Segre D, Vitkup D, Church GM. Analysis of optimality in natural and perturbed metabolic networks (MOMA). *PNAS*. 2002;99(23):15112–15117.
- [20] Shlomi T, Berkman O, Ruppin E. Regulatory on/off minimization of metabolic flux changes after genetic perturbations (ROOM). *PNAS*. 2005;102(21):7695–7700.
- [21] King ZA, Dräger A, Ebrahim A, et al. Escher: a web application for building, sharing, and embedding data-rich visualizations of biological pathways. *PLoS Computational Biology*. 2015;11(8):e1004321.
- [22] Rowe E, Palsson BØ, King ZA. Escher-FBA: a web application for interactive flux balance analysis. *BMC Systems Biology*. 2018;12:84.
- [23] Moretti S, Tran VDT, Mehl F, et al. MetaNetX/MNXref: unified namespace for metabolites and biochemical reactions. *Nucleic Acids Research*. 2021;49(D1):D570–D574.

- [24] King ZA, Lu J, Dräger A, et al. BiGG Models: a platform for integrating, standardizing and sharing genome-scale models. *Nucleic Acids Research*. 2016;44(D1):D515–D522.
- [25] Nogales J, Gudmundsson S, Knight EM, Palsson BØ, Thiele I. Detailing the optimality of photosynthesis in cyanobacteria through systems biology analysis (iJN678). *PNAS*. 2012;109(7):2678–2683.
- [26] Joshi CJ, Peebles CAM, Prasad A. Modeling and analysis of flux distribution and bioproduct formation in *Synechocystis* sp. PCC 6803 using a new genome-scale reconstruction (iSynCJ816). *Algal Research*. 2017;27:295–310.
- [27] Nogales J, Mueller J, Gudmundsson S, et al. High-quality genome-scale metabolic modelling of *Pseudomonas putida* highlights its broad metabolic capabilities (iJN1463). *Environmental Microbiology*. 2020;22(1):255–269.